

分类号: TP391.4

学校代号: 10150

UDC: 004.9 密级: 公开

学 号: 20214062

大连交通大学

专业硕士学位论文

基于协同过滤的推荐系统方法研究

Research on recommendation system methods
based on collaborative filtering

学 生 姓 名 :	俞军
校内导师及职称:	宋旭东 教授
行业产业导师及职称:	姜涛 高级工程师
专 业 名 称 :	计算机技术
研 究 方 向 :	计算机应用技术
论 文 类 型 :	应用研究
培 养 类 型 :	非全日制专业学位硕士
论文答辩日期:	2024 年 8 月 5 日
学位授予单位:	大连交通大学

摘 要

随着信息时代的蓬勃发展，海量信息让人们面临了信息过载的困境，如何在这些海量信息中为用户提供个性化的推荐成为了一项极为重要的任务。在推荐系统领域，协同过滤技术作为一种重要的推荐技术，被广泛应用于各种领域，为用户提供个性化的推荐服务。本文将着重探讨基于协同过滤的推荐系统，展开讨论其原理、算法、实现以及应用，同时分析了其优缺点和改进点以及未来发展趋势。

基于用户的协同过滤推荐系统的核心理念是通过计算用户之间的相似性来进行推荐，从而为用户提供个性化的推荐体验。系统通过分析用户之间的行为和偏好，找出相似的用户群体，并基于这种相似性进行推荐。这种方法在实际应用中得到了验证，证明了其效性，并为用户提供了更加符合其需求的推荐结果。

然而，当前基于用户的协同过滤推荐系统仍面临着一些挑战。比如数据稀疏性、冷启动、推荐准确度以及灵活性差等问题。数据稀疏性指的是用户与物品之间的交互数据较少，导致难以准确计算用户之间的相似性；而冷启动问题则是指对于新用户或新物品，系统无法准确进行推荐。针对以上问题，本文围绕基于用户的协同过滤推荐算法进行了充分研究，主要包含以下工作：

(1) 详细分析了协同过滤技术中的冷启动和稀疏性问题，并深入梳理了现有的解决方案，为后续算法的改进提供了清晰的背景和起点。接着，详细介绍了基于用户的协同过滤算法-UBCF 的核心原理，包括其计算过程和应用场景。为了验证 UBCF 算法的有效性，使用 MovieLens 数据集进行实验，详细记录了实验步骤和结果，并深入分析了该算法的局限性和改进空间。

(2) 在传统基于用户协同过滤的推荐算法基础上进行了创新，提出了一种基于矩阵分解的协同过滤推荐算法-MDCF。MDCF 算法相较于传统方法，除了可以缓解矩阵稀疏性问题，还在计算效率、数据处理规模以及应用便利性上均有所突破。实验结果证明，新算法在提升推荐效果方面表现优异，为推荐系统领域带来了新的可能性。

(3) 针对传统的协同过滤的推荐算法存在的冷启动问题，引入了融合多层感知机的协同过滤推荐算法-MPCF。MPCF 算法能有效解决冷启动问题，它通过利用神经网络的嵌入层，将用户和物品映射到低维稠密向量空间，从而捕获用户和物品的深层特征，提高预测准确率，通过多层非线性变换来学习用户和物品之间的复杂关系，这使得 MPCF 能够更准确地捕捉用户兴趣，并为用户生成更精准的推荐。这一算法的引入，不仅在技术上实现了对前两项工作的超越，通过深度学习的能力，提升了推荐系统的预测性能，而且还增强了推荐的可解释性和灵活性，使得推荐系统能够更好地适应用户行为的动态变化和对推荐多样性的需求。

(4) 在前面的研究基础上, 综合运用协同过滤算法, 设计一个电影推荐系统。通过图形用户界面进行交互, 不仅让用户可以轻松浏览和搜索想看的电影信息, 还能收集用户反馈, 持续优化推荐算法与用户体验。整个系统划分为用户管理和电影管理两大核心模块, 此设计有利于系统资源的合理分配与高效管理, 同时也便于未来功能的增减与升级。

关键词: 协同过滤推荐系统; 冷启动问题; 数据稀疏性; 矩阵分解; 多层感知机

Abstract

In the era of booming information technology, the overwhelming volume of data has led to an overload of information for users, making it an increasingly critical task to provide personalized recommendations amidst this deluge. In the field of recommendation systems, collaborative filtering techniques have emerged as a pivotal approach and have been widely applied across various domains to offer customized recommendation services to users. This thesis focuses on the recommendation system based on collaborative filtering, delving into their principles, algorithms, implementations, applications, and discussing their pros and cons alongside future trends.

The essence of user-based collaborative filtering lies in computing the similarity between users to generate personalized recommendations. The system analyzes user behavior and preferences, identifies clusters of similar users, and bases its recommendations on these similarities. This method has proven effective in practical applications, providing users with more relevant and tailored recommendations.

However, the current user-based collaborative filtering recommendation system still faces some challenges. Data sparsity and cold start are two prominent problems. Data sparsity means that there is less interaction data between users and items, which makes it difficult to accurately calculate the similarity between users. The cold start problem is that the system cannot accurately recommend new users or new items. Future research will focus on solving these problems in order to improve the accuracy, real-time and interpretability of the recommendation system, while paying attention to protecting the privacy of users.

(1) The problems of cold start and sparsity in collaborative filtering technology are analyzed in detail, and the existing solutions are thoroughly sorted out. Then, the core principle of user-based collaborative filtering algorithm UBCF is introduced in detail, including its calculation process and application scenarios. In order to verify the effectiveness of the UBCF algorithm, experiments are carried out using MovieLens dataset, the experimental steps and results are recorded in detail, and the limitations and improvement space of the algorithm are analyzed in depth.

(2) Based on the traditional user-based collaborative filtering recommendation algorithm, a collaborative filtering recommendation algorithm MDCF based on matrix decomposition is proposed. Compared with traditional methods, MDCF algorithm can not only alleviate the problem of matrix sparsity, but also make breakthroughs in computing efficiency, data processing scale and application convenience. The experimental results show that the new algorithm has excellent performance in improving the recommendation effect, which brings new possibilities to the field of recommendation system.

(3) To solve the cold start problem of traditional collaborative filtering recommendation algorithm, MPCF, a collaborative filtering recommendation algorithm integrating multi-layer perceptron, is introduced. The MPCF algorithm can effectively solve the problem of cold start.

By using the embedded layer of neural network, it maps users and items to low-dimensional dense vector space, thereby capturing the deep features of users and items, improving the prediction accuracy, and learning the complex relationship between users and items through multi-layer nonlinear transformation, which enables MPCF to capture users' interests more accurately. And generate more accurate recommendations for users. The introduction of this algorithm not only surpassed the previous two works technically, but also improved the predictive performance of the recommendation system through the ability of deep learning, and also enhanced the interpretability and flexibility of the recommendation system, so that the recommendation system can better adapt to the dynamic changes of user behavior and the demand for the diversity of recommendations.

(4) On the basis of the previous research, a movie recommendation system is designed by comprehensive use of collaborative filtering algorithm. Interaction through the graphical user interface not only allows users to easily browse and search for the movie information they want to see, but also collects user feedback to continuously optimize the recommendation algorithm and user experience. The whole system is divided into two core modules: user management and film management. This design is conducive to the reasonable allocation and efficient management of system resources, and also facilitates the future function increase or decrease and upgrade.

Key Words: Collaborative filtering recommendation system; Cold start problem; Data sparsity; Matrix decomposition; Multilayer perceptron

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.2.1 推荐系统研究现状.....	2
1.2.2 协同过滤研究现状.....	3
1.3 本文研究内容及意义.....	7
1.4 本文结构安排.....	8
本章小结.....	9
第二章 推荐系统及相关技术介绍	11
2.1 推荐系统综述.....	11
2.1.1 推荐系统的定义.....	11
2.1.2 推荐系统的分类.....	11
2.2 推荐系统算法.....	14
2.2.1 协同过滤算法.....	14
2.2.2 基于内容的算法.....	17
本章小结.....	19
第三章 协同过滤推荐算法分析	20
3.1 协同过滤矩阵稀疏性问题分析	20
3.2 协同过滤冷启动问题分析	22
3.3 基于用户的协同过滤算法	23
3.4 实验过程及分析.....	25
3.4.1 实验数据集.....	25
3.4.2 实验步骤.....	26
本章小结.....	30
第四章 基于矩阵分解的协同过滤推荐算法	31
4.1 引言.....	31
4.2 算法的改进点.....	31
4.3 MDCF 算法	32
4.4 实验过程及分析.....	35
4.4.1 实验数据集.....	35
4.4.2 实验步骤.....	35
本章小结.....	40
第五章 融合多层感知机的协同过滤推荐算法	42
5.1 引言.....	42

5.2 神经网络算法分类.....	42
5.3 MPCF 算法.....	45
5.3.1 算法描述.....	45
5.3.2 实验过程及分析.....	47
本章小结.....	52
第六章 系统设计与实现	53
6.1 系统功能结构设计.....	53
6.1.1 用户管理.....	53
6.1.2 电影管理.....	54
6.1.3 推荐算法模块.....	54
6.1.4 系统使用流程.....	55
6.2 系统界面功能实现.....	55
6.2.1 登录注册界面.....	55
6.2.2 用户主界面.....	58
6.2.3 电影分类搜索界面.....	59
6.2.4 电影详情界面.....	59
6.2.5 用户个人界面.....	61
6.3 系统测试.....	61
本章小结.....	63
结论.....	64
参考文献.....	65

第一章 绪论

1.1 研究背景及意义

协同过滤推荐系统是推荐系统领域中一种重要的个性化推荐技术，其研究背景可以追溯到信息过载问题的日益加剧和用户个性化需求的增长。随着互联网和移动互联网的快速发展，用户在日常生活中接触到的信息越来越多，但如何从海量信息中快速准确地找到符合自己需求的内容成为一个挑战。传统的信息检索方式往往无法满足用户的个性化需求，因此推荐系统应运而生。推荐系统在解决信息过载与在线购物和在线内容服务上起着至关重要的作用，其有效性取决于推荐模型或算法对个体用户的独特兴趣和偏好的理解能力。推荐过程是一种用户和系统之间的复杂交互行为^[1]。

推荐系统的出现可以有效解决信息过载问题，为用户提供个性化的信息服务。而在推荐系统中，协同过滤技术作为其中一种核心技术之一，基于用户的协同过滤推荐系统则是其中的重要类型之一。协同过滤技术通过分析用户的历史行为数据和偏好，发现用户之间的相似性，进而推荐给用户其他相似用户喜欢的物品，从而实现个性化的推荐。

基于用户的协同过滤推荐系统具有以下几点重要意义：

首先，可以帮助用户快速准确地找到自己感兴趣的内容，节省用户的时间和精力；其次，可以提高信息的焦点性和相关性，让用户获得更加满意的推荐结果；再者，可以促进信息和资源的有效传播和利用，提升网站或应用的用户满意度和粘性；

最后，可以为电商、新闻、社交媒体等领域提供更好的个性化服务，促进商业的发展和用户体验的提高。例如，美国亚马逊公司的销售收益中，至少有 20% 是通过推荐算法获得的^[2]，如今，快手、抖音等公司将推荐作为其核心，新闻、视频等形式的推荐系统通常会带来 30%~40% 的网络流量或销售增长^[3]。

总的来说，基于协同过滤的推荐系统在信息科技领域具有重要的研究背景和应用意义，为用户提供个性化的推荐服务，促进信息的传播和利用，推动相关领域的发展和创新。本文通过对传统的基于用户的协同过滤算法进行改进，并设计实现了电影推荐系统，旨在提高带给用户的推荐体验。

1.2 国内外研究现状

1.2.1 推荐系统研究现状

推荐系统是融合了数据挖掘、机器学习、预测算法等多种学科组成的一个研究领域。它在电子商务、娱乐等领域起到了关键作用。推荐系统的工作原理主要包括三个基本步骤：数据收集、模型分析和结果推荐。首先，系统收集用户的历史行为数据，如浏览记录、购买记录等，通过分析这些数据，了解用户的兴趣偏好。然后，利用机器学习模型对这些数据进行分析，预测用户可能感兴趣的内容。最后，根据分析结果，向用户推荐个性化的信息或服务。

推荐系统的起源可以追溯到 1992 年，当时施乐帕洛阿尔托研究中心（Xerox PARC）开发了一种基于协同过滤算法的系统，虽然最初是用来过滤垃圾邮件，但它标志着推荐技术的起步，预示着后来广泛应用于个性化推荐领域的重要思想。1994 年被认为是推荐系统成为一个独立研究领域的关键节点，当时美国明尼苏达大学的 GroupLens 研究组推出了 GroupLens 系统，首次提出了基于协同过滤（Collaborative Filtering, CF）来进行推荐的思想，GroupLens 不仅创新性地利用用户之间的共同喜好来预测未知的用户偏好，还建立了推荐问题的形式化模型。这个系统引领了推荐系统今后十几年的发展。GroupLens 团队^[4]在 1997 年首次提出推荐系统（Recommender System）的概念。

后来，基于用户的协同过滤算法、基于物品的协同过滤算法、基于模型的协同过滤算法等逐渐被提出。进入 21 世纪，尤其是互联网泡沫之后的 Web 2.0 时代，推荐系统迎来了飞速发展的时期。在线零售商如 Amazon、Netflix 等纷纷将推荐系统融入其核心业务，通过对海量用户行为数据进行分析，实现商品、电影和其他内容的高度个性化推荐，从而显著提高了用户体验和企业收益。此阶段的特点是大量使用协同过滤、基于内容的过滤等多种算法，并逐渐结合其他领域技术，如矩阵分解、图论等方法提高推荐精度。

随着互联网及相关技术的不断革新和普及，推荐系统在众多行业和领域展现出了巨大的潜力和价值。特别是在电子商务领域，推荐系统的应用已经成为商家提升销售业绩和客户满意度的关键策略之一。通过对用户历史购买记录、浏览行为、搜索关键词、产品评价、甚至社交网络活动等多元数据的深度挖掘与分析，推荐系统能够精确描绘出用户的消费偏好和潜在需求，进而提供高度个性化的商品推荐。例如，服装电商可根据用户的体型、风格、季节性需求等因素，推送针对性的服饰搭配和新品推荐，不仅提升了转化率，也在无形中增强了用户的品牌忠诚度和购物体验。

在电影和音乐流媒体服务领域，推荐系统同样发挥着至关重要的作用。通过算法对用户过去的观影记录、打分、收藏、跳过等情况进行分析，以及对影片和音乐本身的标签、类别、导演、演员、歌手等信息的理解，推荐系统可以准确捕捉到用户的审美倾向和情感需求，从而推荐出符合用户口味的影视作品和音乐曲目。这种个性化推荐不仅有助于用户发现新的内容，增加其在平台上的停留时间，也能促使用户产生更高的付费意愿，从而带动平台收入的增长。

而在新闻资讯和社交网络领域，推荐系统的存在更是大大丰富了信息传播的多样性和有效性。新闻推荐系统依据用户的阅读历史、地理位置、时间段、话题偏好等因素，实现新闻内容的精准推送，让用户在信息海洋中迅速找到自己关心的热点事件和深度报道，从而提升用户的活跃度和粘性。与此同时，在社交网络中，推荐系统不仅能够为用户推荐与其兴趣相符的好友、社群和活动，还有助于建立更紧密、更有价值的人际关系网络，增强用户的社交体验和归属感。

综上所述，推荐系统作为一种先进的信息服务手段，其在电子商务、电影音乐、新闻传媒和社交网络等领域的广泛应用，有力推动了数字经济的发展进程，改变了人们获取信息和服务的方式，促进了各行各业向更高层次的个性化和智能化转型。随着大数据、人工智能和云计算等前沿技术的持续创新，推荐系统的功能将更加完善，应用将更加广泛，无疑将成为未来信息社会中不可或缺的一部分。

近年来，推荐系统继续向纵深方向拓展。一方面，多模态推荐系统逐渐流行，通过整合文本、图像、音频、视频等多种信息源，形成更为立体全面的用户画像；另一方面，实时推荐和动态更新成为新要求，以应对瞬息万变的用户兴趣和市场环境。此外，推荐系统的伦理与社会责任问题凸显，如何在保证推荐效果的同时，兼顾用户隐私保护、减少信息茧房效应、保障推荐公平性等成为了业界关注的焦点。

总结来说，推荐系统带动了不同行业的发展，其发展历程是一段由简至繁、由单一技术向多元化综合技术体系转变的过程，其目标始终围绕着如何更高效、更准确、更人性化地满足用户个性化需求。展望未来，随着大数据、人工智能和云计算技术的持续进步，推荐系统将在更多领域扮演重要角色，并不断优化以解决新的技术和应用挑战。

1.2.2 协同过滤研究现状

早期的推荐系统主要基于简单的规则和人工设定的权重，例如基于内容的推荐，根据物品的属性和用户的喜好进行匹配。这些系统虽然简单，但已经能够在一定程度上解决信息过载的问题。

随着互联网和大数据技术的发展,推荐系统开始采用更复杂的算法和模型。其中,协同过滤(Collaborative Filtering)成为了一种主流的推荐方法。协同过滤利用用户的行为数据(如购买记录、浏览历史等)来预测用户的兴趣,从而为用户推荐可能感兴趣的内容。

1994年,Chrisopher等提出了基于内容的推荐系统,该系统使用用户历史行为数据对用户物品的喜好进行预测,这成为了协同过滤推荐系统的基础。2000年,Epinions.com网站是早期的协同过滤推荐系统之一,它通过分析用户对物品的历史行为(如购买、评分、点击等)为用户生成个性化推荐^[5]。这种方法被称为基于行为的推荐方法。Epinions.com的推荐系统在当时取得了很大的成功,它为用户提供了更为个性化的购物体验,也带动了其他电商平台对于推荐系统的研发和应用。

亚马逊(Amazon.com)网站^[6]在1998年推出了基于物品的协同过滤算法,产生了良好的推荐效果。并在2003年发表的论文中公布了这一算法。但其存在泛化能力差和利用的信息有限两个问题。例如用户统计学特征、物品属性特征和上下文信息等特征都无法引入模型中,造成了有用信息的遗漏^[7]。

然而传统的协同过滤算法仍然存在数据稀疏性、冷启动、灵活性、推荐准确度和多样性^[8]、隐私保护^[9]等问题。首先,用户历史数据矩阵稀疏问题导致在计算相似度的时候出现结果不够准确^[10]的情况。其次,冷启动问题是推荐系统所面临的最大问题之一,也是最备受关注的问题。

很多文献和学者将冷启动问题和数据稀疏问题看成不同问题,这其实是不合理的。冷启动问题可以分为用户冷启动和物品冷启动。用户冷启动指的是新用户刚加入系统时,由于没有足够的历史数据,系统难以对其进行准确的推荐。物品冷启动则是新物品刚加入系统时,由于缺乏用户的评价和反馈,系统难以将其推荐给用户^[11]。

2003年,Church等人在其研究中引入了矩阵分解方法,以解决协同过滤中的冷启动问题。Koren在2009年提出了矩阵分解算法引入了隐向量的概念^[12],增强了模型对矩阵稀疏性问题的处理能力。2007年,Bradley等提出了基于矩阵分解的协同过滤推荐算法^[13]。该算法可以学习用户和物品的隐向量,从而提高推荐的准确性和个性化程度。这种方法在处理冷启动问题和提高推荐效果方面取得了较好的平衡。Rendle在2010年提出了FM(因子分解机)模型^[14],该模型通过两两组合特征进行特征交叉,并利用每个特征的隐权重向量的内积作为特征交叉的权重,这种组合特征的方法不仅有助于学习特征与特征之间的关联性,从而提升推荐性能,还能更有效地解决数据的稀疏性问题。同年,Koren等人在其研究中提出了一种基于随机矩阵分解的方法。该方法结合了协同过滤和机器学习的思想,通过利用随机性对用户-物品交互矩阵进行分解,生成低维的潜在特征表示。它可以更好地处理稀疏数据,并且能够有效地解决冷启动问题。

另外，用户的兴趣会随着时间的推移而改变，而传统推荐算法却没考虑到这一点。国内外学者对上述问题进行了很多研究。比如，沈西挺等考虑到用户间的相似性不仅与用户评过分的项有关，还与用户本身对项目的感兴趣程度有关^[15]，因此基于用户兴趣的相似性计算方式被提出，该方法在一定程度上降低了传统算法中数据稀疏带来的负面影响。

用户协同过滤推荐是一种基于用户行为和偏好的推荐算法。最初的协同过滤算法是基于邻居的方法，通过分析用户之间的相似度来进行推荐。该方法的局限性在于不能很好地应对数据稀疏和冷启动问题。为了解决数据稀疏性问题，研究者们提出了基于矩阵分解的方法^[16]，如经典的 SVD (Singular Value Decomposition)、NMF (Non-negative Matrix Factorization) 和 ALS (Alternating Least Squares)。这些方法通过将用户-物品评分矩阵分解为低维的隐含特征矩阵，来捕捉用户和物品的隐含兴趣。但是，这些方法对于长尾物品推荐效果较差。

近年来，深度学习的兴起为用户协同过滤推荐提供了新的研究方向。研究者们提出了各种基于深度学习的推荐模型，如基于神经网络的协同过滤、深度矩阵分解和注意力机制等。这些模型可以充分运用海量用户行为数据进行端到端的全面训练，并且能够捕捉到更复杂的用户兴趣和物品的特征。2014 年，Chen 等人在其研究中提出了一种结合深度学习和协同过滤的方法^[17]，该方法使用深度神经网络 (Deep Neural Networks, DNN) 来学习用户和物品的隐向量，这些隐向量能够捕捉用户和物品之间复杂的非线性关系，并用于度量用户与物品之间的相似性或匹配程度，从而提高推荐的准确性和个性化程度。

具体来说，该方法首先利用深度学习技术构建用户和物品的隐含特征向量，这些高维且抽象的向量能够捕获并编码用户的行为习惯、个人偏好、情绪反应等多种难以直接观察但又影响其选择行为的潜在因素；同时，对于物品而言，其隐向量则可能蕴含了物品的本质属性、主题内容、流行趋势以及其他不易量化却与用户喜好紧密相关的特性。

在此基础上，通过训练深度神经网络，使得用户和物品之间的交互数据得以高效地映射至对应的隐向量空间中，进而在这个空间里计算用户与物品之间的相似度或者匹配得分。这种方法显著提高了推荐系统预测用户兴趣及潜在喜好的能力，从而使推荐结果更具精确性与个性化特点，有效避免了因过于简单的相似性度量导致的推荐偏差和信息过载问题。

Chen 等人^[17]的研究成果不仅拓展了推荐系统的设计思路和技术边界，而且在实际应用中也取得了显著的效果，证明了深度学习与协同过滤相结合的策略在提高推荐性能方面具有巨大潜力，对于整个推荐系统领域的研究与实践产生了深远的影响。

谷歌在 2016 年的 WideDeep、微软公司的经典深度神经网络模型 Deep Crossing 以及 FNN、PNN 等一批优秀模型的提出^[18]，昭示深度学习的推荐系统时代已经来临。

为了进一步改进推荐的准确性和个性化程度，研究者们引入了更多的上下文信息，如时间、地理位置、社交关系等。基于上下文的推荐可以更好地理解用户的行为和需求，并且能够提供更加精准的推荐结果。

除了算法的研究，推荐系统还面临着隐私保护和公平性等重要问题。近年来，研究者们开始探索如何在推荐系统中合理地处理用户隐私，并通过算法设计来提高推荐的公平性。

例如，Liang 等人^[19]探索了推荐系统领域的一个新颖方向，他们在研究中兼顾了推荐的准确性和多样性，通过项目链接的知识图谱上实施嵌入技术，实现了推荐结果的高精度及丰富的多样性体验。Yan 及其团队^[20]的工作则集中在改进协同过滤推荐算法的多样性上，他们创新性地引入了覆盖粗糙颗粒计算模型。这一策略通过在粗糙颗粒空间内进行粒度的灵活转换，有效增强了推荐列表的多样性，为用户提供了更多元化的选择。Ge 等人^[21]针对推荐系统与用户隐私保护的融合进行了深入探讨，他们不仅构建了一个结合隐私保护机制的推荐算法框架，还前瞻性地提出了“推荐物联网”的理念。该方法通过设立加密的隐私映射集合，巧妙地提升了推荐效率，同时大幅度降低了涉及用户隐私信息处理的计算负担。Bao 等人^[22]则致力于开发一个更加安全的推荐框架，他们创造性地将局部差分隐私技术融入协同过滤推荐算法之中，设计了一套数据扰动策略。这一方案不仅成功掩饰了用户的实际评分数值，还有效隐藏了用户的评分行为模式，实现了用户隐私的双重保护，同时保持了推荐系统的实用性与有效性。

总的来说，用户协同过滤推荐的发展已经从最初的基于邻居的方法逐渐发展到基于矩阵分解、深度学习和上下文信息的方法。未来的研究方向将更加注重解决数据稀疏、长尾推荐、隐私保护和公平性等问题，以提供更好的个性化推荐体验。

例如，一种有效的手段是采用隐私保护技术，包括差分隐私、同态加密、安全多方计算以及近年来兴起的联邦学习等，旨在在数据集中进行模型训练和推荐运算时，确保原始数据不被泄露，只分享经过加密或去标识化的中间结果。其中，基于联邦学习的推荐系统允许各参与方在本地保留数据，仅交换模型参数而非原始数据，从而大大降低了隐私泄露的风险。

另外，推荐系统的公平性问题也越来越受到关注。传统的推荐算法可能会加剧信息过载现象，导致热门内容过度曝光而冷门内容被忽视，形成“富者愈富”的马太效应，影响内容生态的多样性。为了提高推荐系统的公平性，研究者正在开发能够在满足个性化需求的同时，避免推荐结果过于偏颇的新一代算法，如考虑用户长期兴趣和短期新鲜感的平衡、实施全局和局部多样性的策略、以及引入公平性约束等机制。

回顾推荐系统的历史沿革，从最初的基于用户协同过滤（UserCF），即通过寻找用户之间的相似性来进行推荐，逐步演进到基于项目协同过滤（ItemCF）、矩阵分解方法（如 SVD、SVD++、ALS 等），再到如今广泛应用深度学习技术（如神经网络、深度矩阵分解、Autoencoder、Transformer 等）和集成上下文信息（如时间、地点、情境等），推荐系统不断提升了对用户行为复杂性的建模能力。

面向未来，推荐系统的研究将继续深化在以下几个关键方向：

数据稀疏性解决方案：针对用户-商品交互矩阵通常存在的极高维度和极低填充率问题，研究者将寻求更强大的模型表达能力和更高效的稀疏数据处理技术。长尾推荐：强化对长尾商品或内容的推荐能力，打破头部效应，让更多的小众和高质量的商品或内容得到展示和发掘。隐私保护：发展更为先进和实用的隐私保护算法和技术，使推荐系统能在严格遵守隐私法规的前提下提供高质量的个性化服务。公平性与透明度：设计更加公平且可解释的推荐算法，让用户理解推荐背后的原因，同时确保不同用户群体会平等地获得符合其兴趣和价值取向的推荐结果。

综上所述，推荐系统正朝着一个更加完善、可靠、公正和尊重隐私的方向进化，力求在智能化服务的过程中，实现商业价值、用户体验与社会伦理的和谐共生。

1.3 本文研究内容及意义

协同过滤是一种广泛应用的推荐系统算法，其核心在于利用用户和物品之间的相似性来生成推荐。在协同过滤的实践中，用户与物品之间的关系通常通过一个用户-物品矩阵来直观展示和计算，该矩阵的每个元素表示用户对物品的偏好或评分。协同过滤的主要目标是预测用户对未评价物品的偏好，并为用户推荐可能感兴趣的物品。

基于上面所述，本文首先基于用户的协同过滤进行了相似性计算，通过计算用户之间的相似性，找到与目标用户相似的其他用户，以便为目标用户推荐物品。并使用相似用户的评分数据进行预测，为目标用户对未评价物品的偏好进行预测。根据预测的评分，为目标用户生成推荐列表。

然而，传统基于用户协同过滤 (User-based Collaborative Filtering, UBCF) 的推荐系统，尽管在挖掘用户间相似性并据此推荐商品或内容方面取得了一定成效，但仍然面临一系列挑战，制约了其在复杂应用场景下的表现：

(1) 数据稀疏性：在实际场景中，用户对物品的评价或交互记录往往呈现极度稀疏的状态。这意味着，对于大规模的物品集合，大多数用户仅对其中一小部分有过行为记录。这种稀疏性导致难以构建全面且精确的用户兴趣模型，影响推荐的准确度。

(2) 冷启动问题: 新用户或新物品加入系统时, 由于缺乏足够的交互历史, 推荐系统难以对其做出有效推荐。对于用户冷启动, 系统需要在几乎没有历史数据的情况下理解其偏好; 而对于物品冷启动, 则需要在没有用户反馈的情况下评估其潜在受欢迎程度。

(3) 计算效率低下: 随着用户和物品数量的增长, UBCF 算法的计算复杂度迅速上升, 尤其是在计算用户间的相似度矩阵时。这不仅消耗大量计算资源, 还可能导致推荐响应时间过长, 影响用户体验。

(4) 灵活性与适应性不足: 传统的协同过滤方法对用户行为模式变化的适应性较差, 难以实时反映用户的动态偏好变化。此外, 它通常仅基于显式反馈 (如评分), 而忽略了隐式反馈 (如浏览时间、点击率) 的潜在价值。

鉴于以上挑战, 探索和实施改进策略显得尤为重要。为了提升推荐系统的准确性和个性化能力, 本文提出了多种改进算法来对传统的协同过滤算法进行改进。

首先提出了基于矩阵分解的协同过滤推荐算法-MDCF。这一算法通过 Spark 的高效分布式处理能力, 有效统一了不同应用场景中类型多样、分布不一的数据, 提升了算法的普适性和准确性。与传统协同过滤相比, 除了可以缓解数据矩阵稀疏性问题, 该算法计算效率更高, 灵活性和适应性也更强。该算法能显著提高推荐系统的性能, 为推荐系统领域带来了新的可能性。

其次提出了一种融合多层感知机的协同过滤推荐算法-MPCF, 该算法利用神经网络学习用户和物品间的复杂关系, 并将这些关系统一到相同度量标准下。相比传统协同过滤算法, MPCF 具有更强的普适性和合理性, 能够自动学习用户和物品的特征向量, 并通过向量预测评分, 解决了冷启动问题。它所具备的非线性建模能力能够更准确地捕捉用户兴趣, 并为用户生成更精准的推荐。此外, MPCF 算法易于与其他模型融合, 如文本描述、图像特征等, 灵活性大大提升。我们相信, MPCF 算法将为推荐系统领域带来更好的用户体验。

1.4 本文结构安排

本论文共分六章, 各章的主要内容安排如下:

第一章重点阐述了论文的选题背景和研究意义, 概述了推荐系统领域的研究历史与现状, 分析了推荐系统目前面临的主要挑战, 并介绍了本论文的主要研究内容, 最后给出了本文结构安排。

第二章聚焦于推荐系统的理论基础与核心技术, 详尽阐述了推荐系统的定义、分类以及推荐算法, 强调了推荐系统在解决信息过剩问题上的关键作用。本章深入探究

了现今推荐系统中广泛应用的多种推荐算法特性，并对比分析了其优势与局限性。本章旨在为进一步研究推荐系统提供了基础和理论支持。

第三章着重研究协同过滤技术中的冷启动和稀疏性问题，梳理了现有解决方案。本章首先概述了协同过滤算法中存在的稀疏性问题及现有解决方法。然后对基于用户的协同过滤算法（UBCF）的核心算法进行了介绍，并提供了相应的计算公式和应用场景。最后，论文展示了如何使用 MovieLens 数据集进行实验，详细描述了实验步骤，实验结果等，分析了该算法的局限性。

第四章为了解决传统的基于用户系统过滤的推荐算法存在的数据矩阵稀疏性问题，提出了一种基于矩阵分解的协同过滤新算法-MDCF。区别于传统的协同过滤算法，新算法采用最小二乘法进行矩阵分解，不仅能够在评分矩阵非常稀疏的情况下仍然保持较高的推荐准确率，还在计算效率、数据处理规模以及应用的便利性上更强。实验结果证实了新算法在提升推荐效果方面的有效性。

第五章为了解决传统的基于用户系统过滤的推荐算法存在的冷启动问题，提出了融合多层感知机的协同过滤推荐算法-MPCF，该算法不仅能有效缓解冷启动问题，提高预测准确率，并具有良好的可解释性和灵活性。然后列举各类神经网络算法（如 MLP、CNN、RNN/LSTM、自编码器、GAN 等）的基本结构、工作原理和关键数学公式，展示其多样性和广泛的应用背景。接着详述 MPCF 算法的具体原理，并选用 MovieLens 数据集进行实验。最终得出结论，MPCF 算法通过神经网络方式改进传统协同过滤，增强了对用户兴趣和物品属性间复杂关系的捕获能力，提高了推荐准确性，适用于大规模推荐系统场景。

第六章，在前面工作基础上，本文设计实现了一个电影推荐系统，针对目前电影推荐系统很难满足用户的个性化需求，通过该系统，用户只需要提供自己的观影行为和偏好，系统就可以帮助用户快速找到符合自己口味的电影作品，提高用户的观影体验和满意度。

最后对整篇论文进行了全面回顾与总结，提炼了本文的主要创新点、尚存的研究局限性，并对未来需要进一步研究的问题进行了前瞻性的探讨与展望。

本章小结

本章首先介绍了推荐系统的研究背景和意义。接着，回顾了推荐系统的历史发展，特别是协同过滤推荐技术的演进和研究现状。通过对现有文献的梳理和分析，我们面临的挑战和未来的发展方向。

最后，本章详细阐述了本文的研究内容和意义，即针对传统协同过滤算法存在的一些不足之处进行改进，并给出了本文全文的组织结构。

第二章 推荐系统及相关技术介绍

2.1 推荐系统综述

2.1.1 推荐系统的定义

推荐系统 (Recommendation System, RS) 作为一种自动化工具, 旨在为用户与物品之间搭建桥梁。在信息爆炸的时代, 它能够有效地协助用户发掘他们可能感兴趣的内容。该系统通常由三个核心部分组成: 前台的直观展示页面, 用于呈现推荐结果; 后台的日志系统, 负责记录和分析用户行为数据; 以及关键的推荐算法系统, 它基于复杂的算法模型, 生成个性化的推荐内容。

Resnick 和 Varian 在 1997 年对推荐系统给出了一个非形式化的定义, 该定义如今已被广泛引用。他们指出, 推荐系统旨在利用电子商务网站向客户提供商品信息和建议, 从而协助顾客做出购买决策。这些系统模拟了销售人员的角色, 帮助顾客完成整个购买过程, 使得购物体验更为便捷和高效。简而言之, 推荐系统通过提供个性化的推荐服务, 为用户在浩瀚的商品海洋中找到符合其需求的产品, 实现购物流程的智能化与个性化。

2.1.2 推荐系统的分类

推荐系统可以根据不同的维度进行分类, 以下是一些常见的分类方式:

(1) 根据策略分类

基于统计学的推荐 (Statistical-Based Recommendation): 这种方法基于用户的统计特征进行推荐, 它的核心在于利用用户的基本信息, 如年龄、性别、地理位置、职业、收入状况等, 来推断其可能的兴趣和偏好, 通常用于解决冷启动问题。

基于内容的推荐 (Content-based Recommendation): 它是根据用户过去的行为和内容的本身的特性来推荐相似内容, 而非用户的个人信息。具体来说, 系统会分析用户历史浏览、购买、评价过的物品特征, 然后寻找具有类似特征的其他物品进行推荐。例如, 在电影推荐系统中, 如果用户经常观看科幻类电影, 系统就会根据电影的内容标签 (如类型、导演、主演等) 推荐其他科幻类电影给该用户^[23]。

基于协同过滤的推荐 (Based on Collaborative Filtering Recommendation) ^[24]: 该方法是一种统计学上基于用户历史行为和兴趣倾向的个性化推荐技术。该方法通过分析海量用户的行为数据, 识别出具有相似兴趣爱好的用户群组或具有相似被用户喜爱程度的物品集合。系统会根据目标用户的过往行为记录, 找出与其兴趣偏好相近的其他用户或物品, 并推荐这些相似用户喜欢的物品或与目标用户已喜欢物品相似的其他物品。通过这样的方式, 协同过滤推荐实现了精准的个性化推荐服务, 有效拓宽用户发现新内容的视野, 提升用户体验和平台黏性。

混合推荐：综合运用多种推荐策略的方法，旨在结合不同方法的优势，提高推荐的准

确度和个性化程度：这种多元化的推荐方式不仅能够解决冷启动问题，还能够适应不同用户的个性化需求，提升用户体验。

(2) 根据实时性分类

实时推荐 (Real-time Recommendation)：实时推荐系统能够实时捕捉并响应用户的行为和需求变化，实时更新并生成推荐结果，特别适用于对推荐结果时效性要求较高的场景，如新闻推送、实时聊天应用中的内容推荐等，适用于对实时性要求较高的领域。

离线推荐 (Offline Recommendation)：离线推荐系统则采取定期（如每日、每周等固定周期）收集用户数据，然后在此基础上进行大规模的数据处理和模型训练，待模型更新完成后一次性生成一段时间内（如接下来的一天或一周）的推荐结果。这种方法适用于实时性要求相对较低，但数据处理量大、模型计算复杂度高的领域，如电商商品推荐、视频网站内容推荐等。

(3) 根据模型构造方式不同分类

基于模型的推荐 (Model-based Recommendation)：指通过构建并训练机器学习模型来实现个性化推荐的过程。此类推荐系统通常会利用大量的用户行为数据、物品属性数据以及其他相关数据，构建预测模型，以预测用户对未接触过物品的喜好程度，进而推荐最有可能吸引用户兴趣的物品。

基于数据的启发式推荐 (Data-driven Heuristic Recommendation)：这种类型的推荐系统不依赖于复杂的机器学习模型，而是基于一些预定义的规则或者简单的统计分析来进行推荐。例如，可以根据用户历史行为中最频繁出现的物品、最新上线的物品、最热门的物品或者是运营人员设定的特殊规则（如“看过 A 的人还喜欢 B”）来实现推荐。

基于关联规则的推荐 (Data-driven Heuristic Recommendation)：关联规则分析是一种从大量交易数据中发现物品之间的有趣关联或规律的方法，如 Apriori 算法和 FP-Growth 算法。基于关联规则的推荐系统会挖掘出“如果用户购买或喜欢了 X，则可能也会喜欢 Y”的模式，并基于这些规则进行推荐。

基于模型的推荐 (Model-based Recommendation)：这种方法通过构建数学模型或机器学习模型来预测用户对未消费过物品的偏好，然后基于模型预测的结果进行推荐。这种推荐方式能够处理更复杂的数据关系和模式，实现更精准的个性化推荐。其主要包括但不限于：协同过滤推荐、基于内容的推荐混合推荐、混合推荐以及基于深度学习的推荐等。

(4) 根据推荐是否个性化分类

个性化推荐 (Personalized Recommendation)：这类系统根据每个用户的独特兴趣、历史行为、偏好等信息进行推荐，力求提供符合用户个性化需求的内容。例如，协同过滤推荐、基于内容的推荐、基于模型的推荐（如深度学习模型）、基于知识图谱的推荐等，均属于个性化推荐范畴。

非个性化推荐 (Non-personalized Recommendation)：这类系统不考虑用户的个性化需求，而是基于通用的规则、热度、随机性或其他固定的准则进行推荐。例如，热门排行推荐、新品推荐、基于标签的固定分类推荐等，这些推荐结果对所有用户都是一样的，不体现个体差异。

(5) 根据数据源分类

基于人口统计学的推荐 (Demographic-based Recommendation)：这种方法主要依赖于用户的人口统计学特征，如年龄、性别、地理位置、职业、教育水平等基本信息，通过对比和分析具有相似特征的用户群体喜好，进而对目标用户进行推荐。例如，如果数据显示 25-35 岁的女性用户更倾向于购买某种类型的商品，系统就会将此类商品推荐给相同年龄段的女性用户。

基于内容的推荐 (Content-based Recommendation)：这种推荐策略重点关注物品本身的属性和内容信息，如电影的类型、导演、主演、题材、评分等，或者是商品的品牌、价格、颜色、材质等特征。系统通过分析用户过去喜欢的物品内容特征，为其推荐具有相似特征的其他物品。

基于协同过滤的推荐 (Collaborative Filtering Recommendation)：协同过滤是根据用户的行为数据进行推荐的一种常见方法，包括用户协同过滤和物品协同过滤两种方式。用户协同过滤通过找出具有相似行为模式的用户群组，然后推荐其他相似用户喜欢但目标用户还未接触过的物品。物品协同过滤则基于物品之间的相似性，推荐与用户已喜欢物品类似的其他物品。在实践中，协同过滤常常利用用户对物品的评分、购买记录、浏览历史等行为数据来计算相似性。

(6) 其他分类

基于社交网络的推荐 (Social-based Recommendation)：这种推荐系统利用用户在社交网络中的社交关系、朋友圈信息等，来进行个性化推荐。它会考虑用户的社交关系对偏好和兴趣的影响，将朋友的行为和偏好结合到推荐模型中。

基于知识图谱的推荐 (Knowledge Graph-based Recommendation)：其充分利用了知识图谱所蕴含的实体、属性和关系信息，通过推理和模式匹配等方法，发现并推荐与用户兴趣、需求高度相关的实体或信息。这种推荐策略不仅可以用户的历史行为和显式反馈，还可以基于实体间内在的语义关联和上下文信息进行推荐，从而提供更准确、更具解释性和新颖性的推荐内容。

需要注意的是，推荐系统的分类并不是绝对的，很多时候一个推荐系统可能结合了多种策略和方法，以达到更好的推荐效果。同时，随着技术的发展和数据的丰富，推荐系统的分类和策略也在不断演变和拓展。

2.2 推荐系统算法

推荐算法其实本质上是一种信息处理逻辑，当获取了用户与内容的信息之后，按照一定的逻辑处理信息后，产生推荐结果。热度排行榜就是最简单的一种推荐方法，它依赖的逻辑就是当一个内容被大多数用户喜欢，那大概率其他用户也会喜欢。但是基于粗放的推荐往往会不够精确，想要挖掘用户个性化的，小众化的兴趣，需要制定复杂的规则运算逻辑，并由机器完成。

推荐算法主要分为以下几步：

召回：当用户以及内容量比较大的时候，往往先通过召回策略，将百万量级的内容先缩小到百量级。

过滤：对于内容不可重复消费的领域，例如实时性比较强的新闻等，在用户已经曝光和点击后不会再推送到用户面前。

精排：对于召回并过滤后的内容进行排序，将百量级的内容并按照顺序推送。

混排：为避免内容越推越窄，将精排后的推荐结果进行一定修改，例如控制某一类型的频次。

强规则：根据业务规则进行修改，例如在活动时将某些文章置顶。

例如在抖音与快手的分发中：抖音强平台基于内容质量分发，快手轻平台基于社交和兴趣分发，抖音：内容质量 > 关系 > 双向互动。快手：内容质量约等于关系 > 双向互动。抖音基于将内容从小流量开始，其中表现优质的内容将不断的进入更大的流量池中，最终进入推荐池，形成 90 天 + 精品召回池，最终的结果也是优质内容的热度随着时间推移逐渐累积增加，头部内容的集中度很高。

2.2.1 协同过滤算法

协同过滤算法（Collaborative Filtering）是一种十分常用的算法，在很多购物平台中都会用到，此外大量论文和研究都属于这个类别。协同过滤算法有两种类型，分别是基于用户（User）和基于物品（Item）的协同过滤算法。

a. 基于用户的协同过滤算法

步骤一：获得每个用户对各个物品的评价或喜爱程度（可通过浏览、收藏和购买等记录来计算）。

步骤二：依据用户对物品的评价计算出所有用户之间的相似度。（也就是利用用户对所有物品的评价来形容用户）

步骤三：选出与当前用户最相似的 K 个用户。

步骤四：将这 K 个用户评价最高并且当前用户又没有浏览过的 N 个物品推荐给当前用户。

b. 基于物品的协同过滤算法

步骤一：与基于用户的算法一样，获得获得每个用户对各个物品的评价或喜爱程度。收集到的数据将形成一个用户-物品评价矩阵，其中每个单元格代表特定用户对特定物品的评价。

步骤二：依据用户对物品的评价计算出所有物品之间的相似度。（也就是用所有用户对物品的评价来形容物品），常用的相似度计算方法包括余弦相似度、皮尔逊相关系数等。

步骤三：对于当前用户，我们首先需要识别出其评价高的物品，即用户喜好高的物品。

然后，利用步骤二中计算出的物品相似度，我们可以找出与这些喜好高的物品相似度最高的 N 个物品。

步骤四：把这相似度最高的 N 个物品推荐给当前用户^[25]。

一个简单的基于用户的协同过滤算法例子。首先我们根据用户对物品的评价构建出一个用户和物品的关联矩阵，如下表 2.1 所示：

表 2.1 用户和物品的关联矩阵
Table 2.1 An association matrix of users and items

	Item1	Item2	Item3	Item4	Item5	Item6
User1	4	3			5	
User2	5		4		4	
User3	4		5	3	4	
User4		3				5

表中的行是不同的用户，列是不同的物品，单元格 (x, y) 的值则是 x 用户对 y 物品的评价（或喜爱程度）。我们可以把某一行视为某一个用户对所有物品偏好的向量，这样把用户向量化后，就可以计算出每两个用户之间的向量距离了，这里我们使用余弦相似度来作为向量距离。

$$\text{余弦相似度}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.1)$$

然后我们就可以得出所有用户的相似度如下表 2.2 所示，其中值越接近 1 就代表这两个用户越相似。

表 2.2 用户和用户的关联矩阵
Table 2.2 User and user association matrix

	User1	User2	User3	User4	User5	User6
User1	1.00	0.75	0.63	0.22	0.30	0.00
User2	0.75	1.00	0.91	0.00	0.00	0.16
User3	0.63	0.91	1.00	0.00	0.00	0.40
User4	0.22	0.00	0.00	1.00	0.97	0.64
User5	0.30	0.00	0.00	0.97	1.00	0.53

如果我们要为用户 1 推荐物品，则需找出与用户 1 相似度最高的 K 名用户（设 $K=2$ ）评价的物品，并去掉用户 1 评价过的物品，确保推荐的物品对用户 1 来说是新的和未知的。最后按照评价大小排序，如图 2.1 所示为给用户 1 的推荐结果。

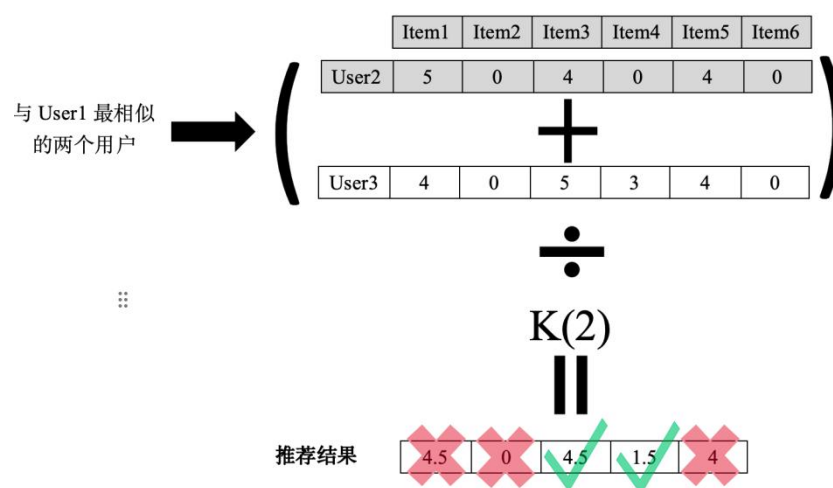


图 2.1 基于用户推荐结果

Fig. 2.1 Based on user recommendations

同样地，接下来举一个简单的基于物品的协同过滤算法例子。

基于物品的计算方式大致相同，把用户和物品的关联矩阵中的某一列视为所有用户对该物品的评价，这样把物品向量化后，就可以计算出物品之间的相似度矩阵。

计算出所有物品之间的相似度，如表 2.3 所示：

表 2.3 物品间相似度
Table 2.3 Similarity between objects

	Item1	Item2	Item3	Item4	Item5	Item6
Item1	1.00	0.27	0.79	0.32	0.98	0.00
Item2	0.27	1.00	0.00	0.00	0.34	0.65
Item3	0.79	0.00	1.00	0.69	0.71	0.18
Item4	0.32	0.00	0.69	1.00	0.32	0.49

最后，如果我们要为用户 1 推荐物品，则需要先找到用户 1 评价最高的物品。然后找到与该物品相似度最高且用户 1 未有评价过的 N（设 $N=2$ ）个物品，将其推荐给用户 1。这样的推荐策略能够确保推荐的物品既符合用户 1 的喜好，又是他未曾接触过的，从而为他带来全新的购物体验。下图 2.2 所示为推荐结果。



图 2.2 基于物品推荐结果
Fig. 2.2 Based on item recommendation results

2.2.2 基于内容的算法

上面提到的算法看起来很好很强大，通过进一步改进也能克服各种缺陷。但是有一个问题是，如果我是《哈利波特》的粉丝，我曾经在商城买过一本《哈利波特与魔法石》。这时商城的书库又上架了一本《哈利波特与死亡圣器》，显然，我应该会很感兴趣。然而上面提到的算法这时候就都不太好使了，因此基于内容的推荐算法就应运而生。

基于内容的推荐算法的基本原理是根据用户的历史行为，获得用户的兴趣偏好，为用户推荐跟他的兴趣偏好相似的标的物，读者可以直观上从图 2.3 理解基于内容的推荐算法^[26]。

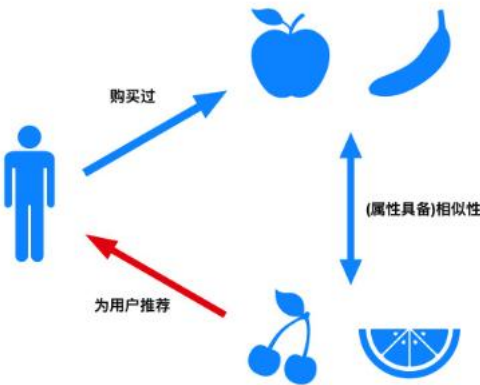


图 2.3 基于内容的推荐算法示意图
Fig. 2.3 Content-based recommendation algorithm schematic

这里举一个简单的例子，现在系统里有一个用户和一条新闻。通过分析用户最近的行为（可以是该用户最近看过什么新闻）和新闻的文本内容，提取出数个关键字，将这些关键字作为属性，也就可以把用户和新闻向量化：

接着计算向量距离，得出用户和该新闻的相似度，再根据相似度进行推荐。在向量化的时候，我们也可以进一步地改进。如，在为一名喜欢看英超联赛的足球迷推荐新闻时，如果有某一条新闻里存在体育、足球、英超关键词，显然前两个词都不如英超这个词来得准确。那么我们如何在系统里突显这种“重要性”呢？这时候我们可以不再简单地用 0,1 或者词频来进行向量化，而是为每个词都赋予权重。这个权重可以从整个新闻语料库中计算出来（如 TF-IDF Term (Frequency-Inverse Document Frequency) 算法），在向量化时引入权重的影响，可以获得更准确的效果。

然而，又有一个问题出现了。要是用户的关键词是足球，而新闻的关键词是德甲、英超，那么按照上面方法计算出的相似度为 0。这显然不符合常理，因为德甲和英超都是足球赛事。为了解决这个问题，我们引入主题聚类的方法，以更深入地理解文本间的语义联系，从而更准确地计算相似度。下图 2.4 所示为主题聚类。

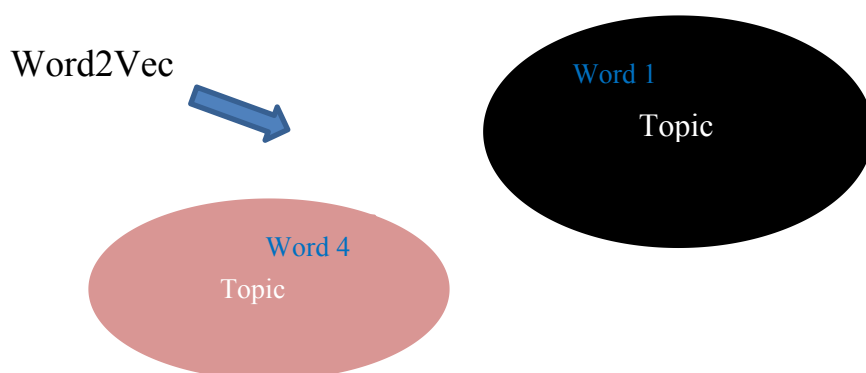


图 2.4 主题聚类

Fig. 2.4 Topic clustering

我们可以通过预训练好的 Word2Vec 对关键词进行表征，然后再聚类出多个主题，最后用主题代替关键词来向量化文本。这样，足球和德甲、英超等关键词就可以通过主题来关联在一起了。

基于内容的推荐算法在多个方面展现了其独特优势：

首先，这种算法对新项目具有良好的适应性。一旦新项目加入系统，因其依赖于项目本身的特征信息而非用户的历史评分数据，算法能够迅速抽取出该项目的特征，并据此对用户做出推荐。这一点在时效性要求极高的新闻推送或实时信息推荐系统中显得尤为重要，无需等待用户先对新项目进行反馈即可实现快速推荐。

其次，针对数据稀疏问题，基于内容的推荐算法展现了解决能力。由于该算法并不依赖于用户对项目的评分矩阵，即使在数据总量较小或者用户与项目交互有限的情况下，依然能够依据项目内容和用户兴趣模型进行个性化推荐。

此外，基于内容的推荐算法具有较高的可解释性。相较于许多难以解读推荐理由的算法，内容过滤算法由于构建了用户兴趣模型，能够清晰地解释推荐结果背后的逻辑，便于用户理解。

虽然能够很好地解决冷启动问题，同时也不会受热度的限制然而，它也会存在一些缺陷，如对文本分析是简单易行，却在大量的网络视频、音频数据分析上存在困难。此外，它推荐的内容都是用户过往的兴趣，无法推荐用户之前没出现但可能喜欢的内容等。

本章小结

本章主要介绍了推荐系统的定义和分类，并详细介绍了推荐系统的算法和原理。推荐系统是一种自动联系用户和物品的工具，旨在信息过载的环境中为用户发现他们感兴趣的信息。推荐系统可以根据策略、实时性、模型构造方式、推荐是否个性化、数据源等不同维度进行分类，其中包括基于统计学的推荐、基于内容的推荐、基于协同过滤的推荐等不同类型。此外，推荐系统算法包括协同过滤算法和基于内容的算法^[27]。协同过滤算法可以基于用户或基于物品进行推荐，通过计算用户间或物品间的相似度来实现精准的个性化推荐。而基于内容的算法则基于用户行为和内容本身的特性来进行推荐，进一步可以引入权重和主题聚类来提高推荐准确度。通过详细介绍推荐系统的定义、分类和算法，本章旨在为进一步研究推荐系统提供了基础和理论支持。

第三章 协同过滤推荐算法分析

3.1 协同过滤矩阵稀疏性问题分析

在现代电子商务、新闻推荐及智能学习等领域中，个性化推荐技术已广泛应用，其中协同过滤技术尤为热门。该技术不受推荐对象限制，能有效发掘用户潜在兴趣并实现兴趣迁移预测。然而，传统协同过滤算法常采用最近邻方法，易受评分矩阵稀疏性问题困扰，若用户间无共同评价项目，相似度难以计算，从而降低推荐效果。

问题描述：例如，推荐系统通常基于用户-物品交互数据构建一个用户-物品评分矩阵，其中行代表用户，列代表物品，元素值表示用户对相应物品的评分或偏好。然而，在现实世界中，由于用户兴趣的多样性和物品数量的巨大，大多数用户只对一小部分物品有过交互（如购买、评价、点击等），导致大部分元素值缺失。这种现象使得用户-物品矩阵极度稀疏。例如，对于一个拥有数百万用户的电子商务网站，每个用户可能只购买过几千分之一的可用商品，因此矩阵的非零元素比例可能远低于 1%。

影响：矩阵稀疏性对推荐系统的影响主要体现在以下几个方面：

(1) 计算效率降低：稀疏矩阵在进行相似度计算、分解等操作时，往往需要处理大量零值，增加了计算复杂度和时间成本。

(2) 模型训练困难：许多推荐算法（如协同过滤）依赖于用户间或物品间的相似性度量，但在稀疏矩阵中，相似性计算的依据不足，可能导致模型难以收敛或学习到有效模式。

(3) 推荐准确性下降：由于缺乏足够的交互信息，推荐系统可能难以准确捕捉用户的个性化需求和物品的潜在关联，导致推荐质量不高。

针对矩阵稀疏性，常见的应对策略包括：

(1) 利用额外信息：引入用户属性（如年龄、性别、地理位置等）、物品属性（如类别、品牌、标签等）或上下文信息（如时间、地点、社交网络关系等）来丰富模型输入，弥补直接交互数据的不足。

(2) 采用适用于稀疏数据的算法：如基于邻域的协同过滤方法（如 UserCF、ItemCF），能够在一定程度上处理稀疏矩阵，通过关注具有共同交互历史的用户或物品来推断喜好。此外，一些矩阵分解技术（如 SVD、SVD++、ALS 等）和深度学习模型（如 Autoencoder、Neural Collaborative Filtering 等）在设计时就考虑了稀疏性问题，能够更有效地处理稀疏矩阵。

(3) 填充缺失值：通过各种手段（如平均值填充、中位数填充、最频繁项填充、基于模型的预测等）对矩阵进行预处理，减少稀疏程度，但需谨慎处理以避免引入噪声或偏差。

(4) 采样和正则化技术：在模型训练阶段，可以采用负采样、正则化项等方式缓解稀疏性带来的问题，提高模型泛化能力。

但是如果只使用用户评分均值或其他默认值填充缺失评分，这并不能准确反映用户对未知项目的偏好。为此，研究者们提出了多种改进措施。Sarwar 等人采用了适用于稀疏数据的算法，运用奇异值分解（SVD）技术有效降低了输入矩阵维度^[28]，尽管其在极端稀疏条件下的预测精度可能不及传统协同过滤算法。

除此以外，研究人员尝试利用 BP 神经网络预测用户对项目的缺失评分以优化评分矩阵，虽需构建复杂的神经网络模型进行离线计算，但仍有助于增加用户间的共同评分项目。在使用 BP 神经网络缓解协同过滤推荐算法的稀疏性问题一文中，张锋、常会友就结合了奇异值分解与最近邻方法，首先采用 SVD 预测未评分项目^[29]，填补原始矩阵，再通过加权相似度计算当前用户的预测评分，实验证明该算法在 EachMovie 数据集上的表现优于 Pearson 算法和 Sarwar 等人的 SVD 方法，有效地解决了数据稀疏性引发的问题。

在基于聚类平滑的可扩展协同过滤这篇文章中则提出了基于 K-means 用户聚类的一种有效矩阵平滑方法^[30]。该算法首先对用户评分习惯进行聚类，利用聚类内平均偏移量预测评分矩阵中的未知评分。其中，用户 u 对未知项目 t 的预填充平滑值计算公为：

$$R_u(t) = \begin{cases} R_u(t) & \text{有评分} \\ \hat{R}_u(t) & \text{未知评分} \end{cases} \quad (3.1)$$

而预填充值的具体计算依据 K-means 聚类结果，遵循公式：

$$\hat{R}_u(t) = \bar{R}_{C_u} + \Delta R_{C_u}(t) \quad (3.2)$$

$$\Delta R_{C_u}(t) = \sum_{u' \in C_u} R_{u'}(t) - \bar{R}_{C_u} / |C_u| \quad (3.3)$$

其中， $C_u(t)$ 代表类 C_u 中所有评价过项目 t 的用户的评分均值，而 $\Delta R_{C_u}(t)$ 由公式 3.3 给出。通过这种方式，原始稀疏矩阵得到了初步平滑。接着，算法利用聚类中心缩小最近邻搜索范围，并在计算邻居相似度时引入权重参数 λ [32]，用于平衡用户真实评分与预测评分的影响。由此，修正后的 K-NN 最近邻算法预测公式变为：

$$R_{u_a}(t) = \bar{R}_{u_a} + \frac{\sum_{i=1}^k w_{ut} \cdot \text{sim}_{u_a, u_i}(t) \cdot (R_{u_i}(t) - \bar{R}_{u_a})}{\sum w_{ut} \cdot \text{sim}_{u_a, u_i}(t)} \quad (3.4)$$

该算法在缓解矩阵稀疏性的同时，通过离线聚类模型提升了在线预测性能，并在 Movie- Lens 和 Each movie^[33]标准数据集上的实验验证了其较现有流行协同过滤算法具有更高的预测精度和更快的运算速度。

此外, 通过结合基于用户和基于项目的协同过滤思想, 在预测缺失评分时同时考虑用户相似性和项目相似性。它确定了相似用户集合 u_a 和相似项目集合 i_k , 预测公式经过调整后包含因子 λ , 使得用户信息和项目信息在预测中可以根据 λ 值动态调整权重, 提高了预测准确性, 但算法整体的时间复杂度较高, 对于系统的可扩展性还需进一步探究。

3.2 协同过滤冷启动问题分析

问题描述: 冷启动问题分为两种情况:

(1) 新用户冷启动: 当推荐系统迎来新注册用户时, 由于缺乏其历史交互数据, 系统无法准确了解用户的兴趣偏好, 难以提供高质量的个性化推荐。

(2) 新物品冷启动: 当系统引入新的商品、电影、文章等物品时, 由于没有用户与其产生过交互, 无法根据已有数据推断其受欢迎程度或与用户偏好的匹配度。

解决冷启动问题的策略包括:

(1) 主动收集用户信息: 通过用户注册时填写问卷、设置偏好、导入社交媒体账号等方式获取初始兴趣信息, 为新用户提供初步的个性化推荐。

(2) 基于内容的推荐: 对于新用户, 可以根据其提供的基本信息 (如人口统计学特征、兴趣标签等) 或对新物品的描述信息 (如类别、标签、文本摘要等), 使用基于内容的推荐方法生成推荐列表。

(3) 群体或领域知识引导: 利用用户群体的共性或特定领域的专家知识, 为新用户提供通用或热门的初始推荐, 待积累一定交互数据后再过渡到个性化推荐。

(4) 探索与利用策略: 在推荐列表中混合已知用户喜好的物品和未知但可能感兴趣的“探索”物品, 平衡短期满意度和长期兴趣发现, 有助于新用户快速找到喜欢的内容。

(5) 社会化推荐: 利用用户的社会关系网络 (如朋友、关注者、共同群体等) 的信息来进行推荐, 特别是对于新用户, 他们的社交圈子往往能提供有价值的兴趣线索。

(6) 迁移学习与预训练: 对于新物品, 可以尝试从相似或相关物品的历史数据中迁移知识, 或者利用预训练模型 (如在大规模数据集上训练的通用推荐模型) 快速初始化新物品的表示, 然后结合少量新数据进行微调。

在处理推荐算法中常见的冷启动问题时, 很多学者提出了各自的解决方案。例如 Hang 等研究者提出了一种富有创意的解决策略^[34], 他们构建了一个包含丰富多属性评分信息的矩阵, 通过深度整合用户与物品的属性数据, 使得推荐系统在迎接新用户或新物品时, 能够依据这些属性信息进行精确匹配。这一方法在实践中取得了显著成效, 有效减轻了冷启动问题带来的困扰。

与此同时, Kim 等专家也在这一领域进行了深入探索^[35]。他们通过对物品特征的细致分析, 巧妙地将用户的偏好信息表达出来, 并引入信息熵的概念, 进一步提升了物品特征识别的精准度。

而在跨域推荐的研究方向上, Liu 等人展现出了独特的洞察力^[36]。他们借助 TrAdaBoost 算法的迁移学习框架, 设计了一种新颖的跨域推荐方法, 为冷启动问题的解决提供了新的思路。

Uyangoda 等研究者则从另一个角度入手^[37], 他们深入挖掘用户与物品的历史交互数据, 利用其中蕴含的用户特征分数关系, 通过评分机制优化推荐算法的输入参数。这一方法不仅提高了预测的准确性, 还降低了对历史用户记录的依赖, 从而有效地应对了冷启动问题。

3.3 基于用户的协同过滤算法

基于用户的协同过滤算法是协同过滤技术中最早出现的一种算法。它的核心假设是: 如果用户在一些项目上的评分表现出较高的相似性, 那么他们在其他项目上的评分也很有可能相似。进一步地, 如果大多数用户对一些项目的评分相似, 那么当前用户对这些项目的评分也可能与其他用户相似。基于这一假设, 该算法通过比较用户间的评分数据来找出兴趣相似的用户群体, 然后利用这些相似用户的评分来预测目标用户可能感兴趣的项目。这种方法在用户兴趣较为固定且用户群体较大的情况下, 能够提供相对准确的推荐结果。

UBCF (User-based collaborative filtering) 是一种方法, 它找到和目标用户兴趣相似的用户集合, 并推荐这些相似用户喜欢的、目标用户没有接触过的物品。例如, 如果 A 和 B 都喜欢科幻电影和书籍, 而 A 看过并喜欢一部新的科幻电影, 那么 UBCF 就会把这部电影推荐给 B。

该算法起始于对用户行为信息的深入分析, 包括用户过去与系统的交互记录以及用户曾经表达过喜好的物品等关键信息。基于这些采集到的用户数据, 算法构建了一个用户物品评分矩阵, 该矩阵反映了用户对各类物品的偏好程度。随后, 算法利用这一矩阵计算用户之间的相似度, 以识别出与目标用户兴趣最为接近的邻居用户集合。最后, 基于这些邻居用户的评分信息, 进行评分预测, 并据此生成推荐列表, 为用户提供个性化的物品推荐。

下图 3.1 所示为基于用户协同过滤推荐算法的主要执行步骤和流程。

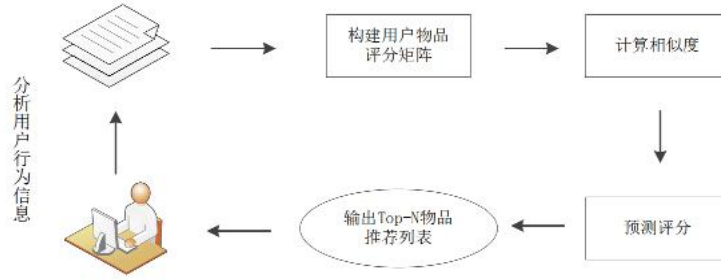


图 3.1 基于用户协同过滤推荐算法流程图

Fig. 3.1 Flow chart of recommendation algorithm based on user collaborative filtering

UBCF 的核心是计算用户之间的相似度，通常使用皮尔逊相关系数或余弦相似度或杰卡德相关系数作为相似度度量。给定用户 u 和用户 v ，令 $N(u)$ 表示用户 u 曾经有过正反馈的物品集合，令 $N(v)$ 为用户 v 曾经有过正反馈的物品集合。那么，皮尔逊相关系数可以定义为：

$$\text{sim}(u, v) = \frac{\sum_{i \in N(u) \cap N(v)} (r_{ui} - \bar{r}_u) (r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in N(u) \cap N(v)} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in N(u) \cap N(v)} (r_{vi} - \bar{r}_v)^2}} \quad (3.5)$$

其中， r_{ui} 表示用户 u 对物品 i 的评分， $\bar{r}_u$ 表示用户 u 对所有物品的平均评分。根据用户之间的相似度，可以为目标用户预测他对某个物品的评分。预测公式如下：

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in S(u, K) \cap N(i)} \text{sim}(u, v) (r_{vi} - \bar{r}_v)}{\sum_{v \in S(u, K) \cap N(i)} |\text{sim}(u, v)|} \quad (3.6)$$

其中， $\hat{r}_{ui}$ 表示预测评分， $S(u, K)$ 表示和用户 u 最相似的 K 个用户集合， $N(i)$ 表示对物品 i 有过评分的用户集合， $\text{sim}(u, v)$ 为用户 u 和 v 的相似度。

杰卡德相关系数 (Jaccard Index)，也被称为杰卡德相似系数，是一种用于比较有限样本集之间相似性与差异性的度量。其定义基于两个集合的交集与并集的比值。具体地说，给定两个集合 A 和 B ，杰卡德相关系数 $J(A, B)$ 的计算公式如下：

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \quad (3.7)$$

其中 $|A|$ 表示集合 A 中元素的个数，即集合 A 的基数， $|B|$ 表示集 B 中元素的个数，即集 B 的基数。

杰卡德相关系数的值介于 0 和 1 之间。当杰卡德相关系数等于 1 时，表示两个数据集完全相同；当杰卡德相关系数等于 0 时，表示两个数据集没有任何共同元素。杰卡德系数越高，两个样本的相似度越高。

Jaccard 系数相关的指标叫做 Jaccard 距离，用于描述集合之间的不相似度。杰卡德距离用两个集合中不同元素占有所有元素的比例来衡量两个集合的区分度，Jaccard 距离越大，样本相似度越低。公式定义如下：

$$d_j(A, B) = 1 - J(A, B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} = \frac{A \Delta B}{|A \cup B|} \quad (3.8)$$

举个例子：

比较两部手机的差异，我们可以用一个 n 维向量来描述这部手机，比如手机有这些特征：1、是否国产 2、是否刘海屏 3、是否安卓系统 4、价格超过 5000。我们用 1 来表示“是”，用 0 来表示“否”，那么：

手机 A 的向量为： $\{1, 1, 1, 0\}$ ，手机 B 的向量为： $\{0, 1, 0, 1\}$ 。

这个向量的各个维度数据值可以当作一个集合，A 和 B 的杰卡德系数为： $J(A,B)=\text{同维度且维度值相同的个数}/\text{总维度数}=1/4$ 。

Jaccard 相似度的缺点是只适用于二元数据的集合。

然而，这种系数具有计算方式简单、处理速度快的优点，因此常被用于处理大规模数据，并广泛应用于信息检索、文本分类、社交网络分析、推荐系统等领域。在这些应用中，通常将相似度高于某个阈值的数据集视为相似的。

对于个性化推荐来说，最核心、重要的算法是相关性度量算法。相关性从网站对象来分，可以针对商品、用户、旺铺、资讯、类目等等，从计算方式看可以分为文本相关性计算和行为相关性计算，具体的实现方法有很多种，最常用的方法有余弦夹角 (Cosine) 方法、杰卡德 (Jaccard) 方法等。Google 对新闻的相似性计算采用的是余弦夹角，CBU 的个性化推荐以往也主要采用此方法。从 9 月份开始，CBU 个性化推荐团队实现了杰卡德计算方法计算文本相关性和行为相关性，并且分别在线上做了算法效果测试。

总的来说，杰卡德相关系数是一种有效的工具，可以帮助我们量化地理解和比较不同样本集之间的相似性和差异性。

3.4 实验过程及分析

3.4.1 实验数据集

个性化推荐中，电影推荐研究时常使用 movielens 上的数据集。该网站的数据集主要分两部分，一是用于推进最新研究进展的数据集。当前最新的是发布于 2019 年 12 月份的 25M 数据集。二是用于高校、组织科研的数据集。该类数据集按其是否带有标签、时间先后、数据集大小分成 6 种数据集。

MovieLens 数据集下载地址：<https://grouplens.org/datasets/movielens/>，此数据集 (ml-25m) 用于电影推荐服务，其中包含了 25000095 个评分和 1093360 个标签，横跨了 62423 部电影^[38]。该数据集中包含了 162541 个用户，时间跨度从 1995-01-09 至 2019-11-21，每个用户至少对 20 部电影进行了评分。

我们下载的是 ml-latest-small.zip，数据量小，便于我们单机使用和运行。

3.4.2 实验步骤

1. 读入用户评分；2. 计算用户相似性；3. 根据用户相似性排序选择前 K 个邻居；4. 根据 K 个邻居的评分项目预测用户评分；5. 选择预测评分最高的前 N 个项目作为推荐列表。

实验参数设置：

相似度计算：选择了余弦相似度作为计算用户之间相似度的方法。

邻居数量：为了生成推荐列表，确定了最近邻居的数量，通常选择 10 或 20 个最近邻居。

评分预测阈值：设定了评分预测阈值，例如，低于 3.0 的预测评分将不会被考虑为推荐。

代码实现：

读取数据文档如图3.2所示：

```
用户id,物品id,评分,timestamp
1,1,4.0,964982703
1,3,4.0,964981247
1,6,4.0,964982224
1,47,5.0,964983815
1,50,5.0,964982931
```

图 3.2 数据集

Fig. 3.2 Data Set

建立关系矩阵 rating 如图 3.3 所示：

物品id	1	2	3	4	...	193583	193585	193587	193609
用户id	...	...	...	...	...	...	...	...	...
1	4.0	0.0	4.0	0.0	...	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
5	4.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...
606	2.5	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
607	4.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
608	2.5	2.0	2.0	0.0	...	0.0	0.0	0.0	0.0
609	3.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
610	5.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0

[610 rows x 9724 columns]

图 3.3 关系矩阵 rating

Fig. 3.3 Relational matrix rating

获取关系矩阵如图3.4所示:

```
[[4.  0.  4.  ... 0.  0.  0. ]
 [0.  0.  0.  ... 0.  0.  0. ]
 [0.  0.  0.  ... 0.  0.  0. ]
 ...
 [2.5 2.  2.  ... 0.  0.  0. ]
 [3.  0.  0.  ... 0.  0.  0. ]
 [5.  0.  0.  ... 0.  0.  0. ]]
```

图 3.4 关系矩阵数组

Fig. 3.4 Relational matrix array

各类相似度计算函数的构建:

(1) 欧式距离

$$d = \sqrt{x_1 - x_2^2 + (y_1 - y_2)^2} \quad (3.9)$$

打印结果如下:

59.11429607125505

(2) 曼哈顿距离

$$d = |x_1 - x_2| + |y_1 - y_2| \quad (3.10)$$

打印结果如下:

862.0

(3) 余弦相似度

$$d = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| * |\vec{b}|} \quad (3.11)$$

打印结果如下:

0.002251389028959687

(4) 皮尔逊相关系数

$$r_{pb} = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 * \sum (y - \bar{y})^2}} \quad (3.12)$$

打印结果如下:

0.004555585466518574

当然我们也可以使用 Python 的 scipy 库来计算两个向量 u_1 和 u_2 之间的皮尔逊相关系数。

打印结果如下：

0.004555585466520217

(5) 使用余弦相似度计算物品相似度矩阵

双重循环计算每一对用户之间的余弦相似度,如下图3.5所示。不过这个速度很慢,然后将相似度矩阵转换为DataFrame。

	÷ 0	÷ 1	÷ 2	÷ 3	÷ 4	÷ 5	÷ 6	÷ 7	÷ 8	÷ 9	÷ 10	÷ 11
0	1.00000	0.02728	0.05972	0.19439	0.12908	0.12815	0.15874	0.13697	0.06426	0.01688	0.13250	0.01646
1	0.02728	1.00000	0.00000	0.00373	0.01661	0.02533	0.02759	0.02726	0.00000	0.06745	0.04442	0.00000
2	0.05972	0.00000	1.00000	0.00225	0.00502	0.00394	0.00000	0.00494	0.00000	0.00000	0.00000	0.00000
3	0.19439	0.00373	0.00225	1.00000	0.12866	0.08849	0.11512	0.06297	0.01136	0.03116	0.05477	0.04994
4	0.12908	0.01661	0.00502	0.12866	1.00000	0.30035	0.10834	0.42908	0.00000	0.03061	0.18381	0.05886
5	0.12815	0.02533	0.00394	0.08849	0.30035	1.00000	0.07584	0.37049	0.01390	0.02038	0.22023	0.09606
6	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	0.00000	0.00000	0.00000	0.00000	0.00000
7	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	0.00000	0.00000	0.00000	0.00000
8	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	0.00000	0.00000	0.00000
9	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	0.00000	0.00000
10	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	0.00000
11	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000

图 3.5 双重循环使用余弦相似度计算的用户相似度矩阵

Fig. 3.5 The user similarity matrix calculated using cosine similarity is doubly cycled

当然，也可以直接调库实现。如下图3.6所示：

	÷ 0	÷ 1	÷ 2	÷ 3	÷ 4	÷ 5	÷ 6	÷ 7	÷ 8	÷ 9	÷ 10	÷ 11
0	1.00000	0.02728	0.05972	0.19439	0.12908	0.12815	0.15874	0.13697	0.06426	0.01688	0.13250	0.01646
1	0.02728	1.00000	0.00000	0.00373	0.01661	0.02533	0.02759	0.02726	0.00000	0.06745	0.04442	0.00000
2	0.05972	0.00000	1.00000	0.00225	0.00502	0.00394	0.00000	0.00494	0.00000	0.00000	0.00000	0.00000
3	0.19439	0.00373	0.00225	1.00000	0.12866	0.08849	0.11512	0.06297	0.01136	0.03116	0.05477	0.04994
4	0.12908	0.01661	0.00502	0.12866	1.00000	0.30035	0.10834	0.42908	0.00000	0.03061	0.18381	0.05886
5	0.12815	0.02533	0.00394	0.08849	0.30035	1.00000	0.07584	0.37049	0.01390	0.02038	0.22023	0.09606
6	0.15874	0.02759	0.00000	0.11512	0.10834	0.07584	1.00000	0.11488	0.09946	0.13210	0.19468	0.04523
7	0.13697	0.02726	0.00494	0.06297	0.42908	0.37049	0.11488	1.00000	0.00000	0.02391	0.23533	0.04619
8	0.06426	0.00000	0.00000	0.01136	0.00000	0.01390	0.09946	0.00000	1.00000	0.04096	0.00000	0.00000
9	0.01688	0.06745	0.00000	0.03116	0.03061	0.02038	0.13210	0.02391	0.04096	1.00000	0.02728	0.09116
10	0.13250	0.04442	0.00000	0.05477	0.18381	0.22023	0.19468	0.23533	0.00000	0.02728	1.00000	0.03190
11	0.01646	0.00000	0.00000	0.04994	0.05886	0.09606	0.04523	0.04619	0.00000	0.09116	0.03190	1.00000

图 3.6 调库使用余弦相似度计算的用户相似度矩阵

Fig. 3.6 The tuning library uses cosine similarity to calculate the user similarity matrix

(6) 构建推荐指数函数计算公式：

$$r(x, y) = \frac{\sum_{y \in N(x, i)} S_{xy} \cdot r_{yi}}{\sum_{y \in N(x, i)} S_{xy}} \quad (3.13)$$

r_{xi} : 预测用 x 对物品 i 的评分

r_{yi} : 预测用户 y 对物品 i 的评分

S_{xy} : 用户 x 和用户 y 之间的相似度

$N(x; i)$: 与用户 x 相似的用户对物品 i 的评分集合构建一个基于用户的推荐
打印结果如下:

推荐得分为 4.498392154133329

(7) 构建预测函数

预测函数的功能: 传入相似度矩阵, 通过对每个用户和每个物品进行计算, 计算出一个推荐矩阵。如下图3.7所示:

	÷ 0	÷ 1	÷ 2	÷ 3	÷ 4	÷ 5	÷ 6	÷ 7	÷ 8	÷ 9	÷ 10	÷ 11
0	0.00000	2.50378	0.00000	0.00000	2.49865	0.00000	3.00000	0.00000	0.00000	3.21833	3.00000	2.00000
1	3.54322	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
2	4.00000	4.00000	4.52153	0.00000	3.00000	3.97868	0.00000	0.00000	0.00000	4.00000	0.00000	0.00000
3	4.08797	2.67389	3.00000	0.00000	0.00000	3.44138	1.00000	0.00000	0.00000	2.76479	3.00000	0.00000
4	0.00000	3.31763	3.00000	0.00000	3.30364	3.00000	3.93007	0.00000	0.00000	3.00485	3.52829	0.00000
5	4.80303	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
6	0.00000	2.49880	2.00000	0.00000	0.00000	4.18226	0.00000	0.00000	0.00000	3.78478	4.51704	0.00000
7	4.51141	0.00000	3.00000	0.00000	3.00000	3.00000	3.49629	0.00000	0.00000	0.00000	0.00000	0.00000
8	3.68573	2.47223	0.00000	0.00000	0.00000	4.50000	0.00000	0.00000	0.00000	4.00908	3.50000	0.00000
9	4.06957	2.83932	2.00000	0.00000	1.74594	4.00000	1.46631	0.00000	0.00000	3.75148	3.69946	0.00000
10	4.49582	4.00000	3.49612	0.00000	0.00000	0.00000	3.49582	0.00000	0.00000	0.00000	5.00000	0.00000
11	3.32592	2.74978	3.25998	0.00000	2.98867	0.00000	4.49639	0.00000	0.00000	2.74978	2.85760	0.00000

图 3.7 推荐矩阵

Fig. 3.7 Recommendation matrix

(8) 构建最终的 Topk 推荐函数

如图3.8所示, 返回排序后的前几个值, 每个用户将获得前三个推荐条目。

	÷ 用户id	÷ 物品id	÷ 推荐指数
0	1	3022	5.00000
1	1	1243	5.00000
2	1	1235	5.00000
3	2	1059	5.00000
4	2	1917	5.00000
5	2	35836	5.00000
6	3	940	5.00000
7	3	1025	5.00000
8	3	661	5.00000
9	4	4291	5.00000

图 3.8 每个用户的 Top-3 推荐列表

Fig. 3.8 Top-3 recommendation list for each user

通过上述的实验步骤及得到的结果, 我们不难发现, 每次计算用户之间相似度时, 需要遍历所有用户及其评分; 在网站用户增长到一定程度后, 计算用户之间的相似度过程将会变得越发复杂和困难; 此外, 基于用户的协同过滤方法数学性不是很好 (即

不是很容易从数学原理的角度去解释)；用户口味变化极快，不是静态的，所以兴趣迁移问题很难反应出来；数据稀疏，用户和用户之间有共同消费行为实际上比较少，而且一般是热门物品，对发现用户兴趣帮助不大。

本章小结

本章对协同过滤中存在的矩阵稀疏性问题和冷启动问题进行了分析，为了应对稀疏性问题，研究者采用了多种策略，包括但不限于：引入额外信息以增强模型输入，如用户属性、物品属性和上下文信息；使用适合稀疏数据的算法，如基于邻域的协同过滤（UserCF、ItemCF）和矩阵分解技术（SVD、SVD++、ALS）以及深度学习模型（Autoencoder、Neural Collaborative Filtering）；对缺失值进行填充，如使用平均值、中位数或基于模型预测；应用采样和正则化技术来改善模型训练和泛化能力。

针对协同过滤的冷启动问题，本文列举了多条解决方案，包括主动收集用户信息、基于内容推荐、群体或领域知识引导、探索与利用策略、社会化推荐和迁移学习与预训练等方法。

基于用户的协同过滤算法（UBCF）的核心是计算用户之间的相似度，并利用相似用户的评分来预测目标用户对未评分项目的偏好。论文介绍了几种常用的相似度计算方法，如皮尔逊相关系数、余弦相似度和杰卡德相关系数，并提供了相应的计算公式和应用场景。

最后，本章展示了针对传统的基于用户的协同过滤算法如何使用 MovieLens 数据集进行实验，详细描述了实验步骤，包括读取用户评分、计算用户相似性、预测评分并生成推荐列表，并演示了如何用 Python 实现几种不同的相似度计算函数。

第四章 基于矩阵分解的协同过滤推荐算法

4.1 引言

基于用户的协同过滤，简而言之，就是基于用户之间的相似兴趣或共同喜好来为他们推荐可能感兴趣的信息。其核心思想在于利用用户对各类物品的评分矩阵，在广泛的用户群体中寻找与指定用户兴趣相似的用户群体。之后，结合这些相似用户对某一物品评价，我们可以预测指定用户对该物品的喜好程度。这种方法在推荐系统中被广泛应用，能够有效地为用户提供个性化的推荐服务。

在基于用户的协同过滤方法中，如果两个用户表现出相似的偏好（即对相同物品的偏好大体相同），那就认为他们的兴趣类似。要对他们中的一个用户推荐一个未知物品，便可选取若干与其类似的用户并根据他们的喜好计算出对各个物品的综合得分，再以得分来推荐物品。其整体的逻辑是，如果其他用户也偏好某些物品，那这些物品很可能值得推荐。

同样也可以借助基于物品的方法来做推荐。这种方法通常根据现有用户对物品的偏好或是评级情况，来计算物品之间的某种相似度。这时，相似用户评级相同的那些物品会被认为更相近。一旦有了物品之间的相似度，便可用用户接触过的物品来表示这个用户，然后找出和这些已知物品相似的那些物品，并将这些物品推荐给用户。同样，与已有物品相似的物品被用来生成一个综合得分，而该得分用于评估未知物品的相似度。

本章将针对上一章节中采用传统的基于用户协同过滤算法实验效果表现不佳，在数据矩阵稀疏问题方面不能很好处理的情况下，提出了基于矩阵分解的协同过滤推荐算法-MDCF，实验结果显示除了可以缓解数据矩阵稀疏问题，还在计算效率、数据处理规模以及应用便利性上均有所突破，推荐准确率明显提升。

4.2 算法的改进点

本章所改进的协同过滤推荐算法相对于普通的协同过滤算法在多个方面进行了显著的提升：

计算效率与扩展性：MDCF 算法利用了 Spark 的分布式计算能力，使得在大规模数据集上运行协同过滤算法成为可能。通过分布式处理，该算法可以显著提高计算效率，并轻松处理海量数据。此外，它还提供了良好的扩展性，可以根据计算资源的需求进行弹性扩展。

准确性：MDCF 算法通过结合矩阵分解技术，可以更好地捕捉用户和物品之间的潜在关系，解决矩阵稀疏问题，提高推荐的准确性。

支持隐式反馈：在真实世界的应用中，隐式反馈（如点击、浏览、购买等行为）往往比显式反馈（如评分、评论等）更为常见。MDCF 算法支持隐式反馈数据的处理，能够根据用户的隐式行为来推断用户的偏好，并生成相应的推荐。

4.3 MDCF 算法

MDCF 算法是建立在 Spark 的基础上，运用矩阵分解技术改进协同过滤算法。推荐模型库当前只包含基于矩阵分解（matrix factorization）的实现，这些模型在协同过滤中的表现十分出色，由此我们也将重点关注这类模型。

Spark 中的机器学习库，集合了众多常见的机器学习算法，如分类、回归、聚类以及协同过滤等^[39]。这使得开发者能够在熟悉 Spark 编程模型的基础上，利用其强大的数据处理能力来执行机器学习 workflow。

在 MDCF 算法中，用户和商品通过一小组隐性因子进行表达，并且这些因子也用于预测缺失的元素。我们可以使用交替最小二乘法（ALS）来学习这些隐性因子。这对于处理用户-物品评分数据和生成个性化推荐非常有效。

(1) 协同过滤参数

MDCF 算法使用交替最小二乘法（alternating least squares, ALS）学习隐藏因子。算法中的参数如下：

numBlocks: 并行计算的 block 数（-1 为自动配置）。

rank: 模型中隐藏因子数。

iterations: 算法迭代次数，通常会在 20 次或更短的迭代中收敛到一个合理的解决方案。

lambda: 正则化参数。

implicitPrefs: 使用显示反馈变量或隐式反馈。

alpha: 隐式反馈变量的参数，用于控制偏好观察中的基线置信度。

(2) 矩阵分解

用户对物品的打分行为可以表示成一个评分矩阵 $A(m \times n)$ ，表示 m 个用户对 n 各物品的打分情况。如下表 4.1 所示：ALS（交替最小二乘法）是一种常用的矩阵分解方法，用于处理评分矩阵中的缺失值问题。评分矩阵 A 的元素 $A(i, j)$ 表示用户 $user_i$ 对物品 $item_j$ 的打分。由于用户不会对所有物品打分，矩阵 A 中存在许多缺失值，即“？”所表示的情况。

表 4.1 用户对物品的评分矩阵

Table 4.1 The user's bisection matrix of items

UIV	v1	v2	v3	v4
u1	4	3	?	5
u2	?	5	4	5
u3	?	?	3	3
u4	5	5	3	3
u5	2	1	5	?

这里采用一种支持不完整评分矩阵的矩阵分解方法，避免了对评分矩阵进行估值填充。在推荐系统中，我们希望得到用户对所有物品的打分情况，因此需要对缺失值进行预测，这称为“矩阵补全”。

ALS 的核心假设是：打分矩阵 A 是近似低秩的，即一个 $m \times n$ 的打分矩阵 A 可以用两个小矩阵 $U (m \times k)$ 和 $V (n \times k)$ 的乘积来近似：

$$A \approx UV^T, k \ll m, n \quad (4.1)$$

其中， U 和 V 是通过优化过程得到的矩阵， k 是一个远小于 m 和 n 的正整数，表示隐因子的维度。通过这种方法，ALS 可以对缺失的评分进行预测，并为用户推荐可能感兴趣的物品。

我们使用用户喜好特征矩阵 $U (m \times k)$ 中的第 i 个用户的特征向量 u_i ，和产品特征矩阵 $V (n \times k)$ 第 j 个产品的特征向量 v_j 来预测打分矩阵 $A (m \times n)$ 中的 a_{ij} 。我们可以得出以下的矩阵分解模型的损失函数为：

$$C = \sum_{(i,j) \in R} [(a_{ij} - u_i v_j^T)^2 + \lambda (u_i^2 + v_j^2)] \quad (4.2)$$

有了损失函数之后，下面就开始介绍优化方法。通常的优化方法分为两种：交替最小二乘法 (alternative least squares) 和随机梯度下降法 (stochastic gradient descent)。本章所提出的 MDCF 算法就是使用交替最小二乘法 (ALS) 来最优化损失函数。算法的思想就是：我们先随机生成然后固定它求解，再固定求解，这样交替进行下去，直到取得最优解 $\min()$ 。因为每步迭代都会降低误差，并且误差是有下界的，所以 ALS 一定会收敛。但由于问题是非凸的，ALS 并不保证会收敛到全局最优解。但在实际应用中，ALS 对初始点不是很敏感，是否全局最优解造成的影响并不大。

(3) 显隐式反馈

MDCF 算法利用交换最小二乘解决矩阵分解问题分两种情况：数据集是显式反馈和数据集是隐式反馈。

推荐系统依赖不同类型的输入数据，最方便的是高质量的显式反馈数据（例如用户评星数据），但是显式反馈数据不一定总是找得到，因此推荐系统可以从更丰富的隐式反馈信息中推测用户的偏好。隐式反馈类型包括购买历史、浏览历史、搜索模式甚至鼠标动作。例如，购买同一个作者许多书的用户可能喜欢这个作者。

了解隐式反馈的特点非常重要，因为这些特质使我们避免了直接调用基于显式反馈的算法。最主要的特点有如下几种：

没有负反馈。即很难推测哪个商品用户不喜欢。这在显式反馈算法中并不存在，因为用户明确告诉了我们他喜欢哪些他不喜欢哪些。

隐式反馈是内在的噪音。虽然我们拼命的追踪用户行为，但是我们仅仅只是猜测他们的偏好和真实动机。例如，因为这个商品可能作为礼物被购买而用户并不喜欢它。

显示反馈的数值值表示偏好（preference），隐式反馈的数值值表示信任（confidence）。基于显示反馈的系统用星星等级让用户表达他们的喜好程度。基于隐式反馈的数值值描述的是动作的频率，例如用户购买特定商品的次数。一个较大的值并不能表明更多的偏爱。但是这个值是有用的，它描述了在一个特定观察中的信任度。

接下来，我们对 MDCF 算法的复杂度进行分析。

MDCF 算法通过矩阵分解技术降低了数据稀疏性的影响，并提高了计算效率。然而，其复杂度分析对于理解其在实际应用中的性能至关重要。

时间复杂度：MDCF 算法的时间复杂度主要取决于矩阵分解的迭代次数和矩阵的规模。假设用户-物品矩阵的大小为 $m \times n$ ，迭代次数为 k ，则 MDCF 算法的时间复杂度大致为 $O(kmn)$ 。

空间复杂度：MDCF 算法需要存储用户-物品矩阵、分解后的用户矩阵和物品矩阵，以及相关的中间变量。因此，其空间复杂度为 $O(m^2 + n^2 + mn)$ ，其中 m 和 n 分别为用户和物品的数量。

评价隐式反馈推荐系统需要合适的手段。

下图 4.1 所示为本文提出的 MDCF 算法采用 ALS 对协同过滤进行优化的算法的流程图，该图描述了 MDCF 算法的基本工作流程，主要分为三个阶段：数据加载与预处理、参数调优与模型训练、预测与推荐。整个流程旨在构建一个高质量的协同过滤推荐系统，通过不断优化参数和模型，提高推荐的准确性。

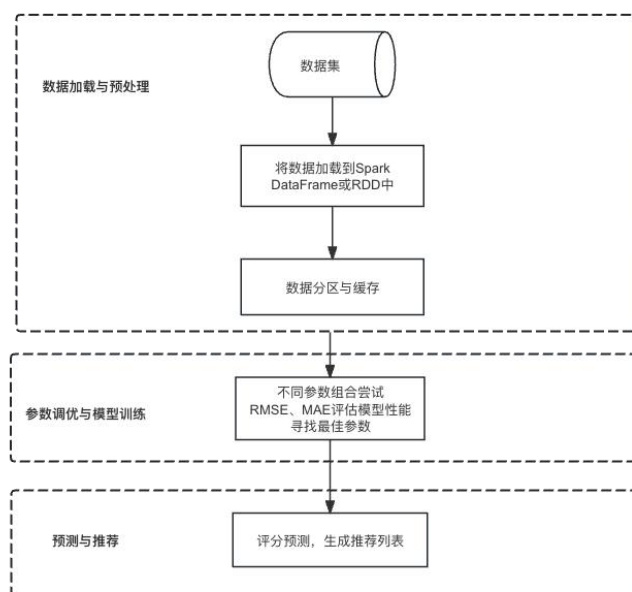


图 4.1 MDCF 算法的流程图

Fig. 4.1 Flow chart of MDCF algorithm

4.4 实验过程及分析

4.4.1 实验数据集

我们使用 GroupLens Research (<http://grouplens.org/datasets/movielens/>) 提供的数据进行实验。该数据为一组从 20 世纪 90 年代末到 21 世纪初由 MovieLens 用户提供的电影评价数据, 包括评分、电影元数据 (如风格类型和年代), 以及关于用户的人口统计学数据 (如年龄、邮编、性别和职业等) [40]。根据不同需求, GroupLens Research 提供了不同大小的样本数据, 包含了评分、用户信息和电影信息三种数据。下面先来看看待处理的电影评价数据, 再给出应用程序并进行分析。

4.4.2 实验步骤

(1) 数据说明:

实验采用的数据是 movielens 中的 ml-100k 版本数据集中的 u.data, 它是用户、电影、评分的数据, 其中用户有 943 人, 电影有 1682 部, 评价有 100000 条。下图 4.2 所示为 u.data 部分数据:

196	242	3	881250949
186	302	3	891717742
22	377	1	878887116
244	51	2	880606923
166	346	1	886397596
298	474	4	884182806
115	265	2	881171488
253	465	5	891628467
305	451	3	886324817
6	86	3	883603013
62	257	2	879372434
286	1014	5	879781125
200	222	5	876042340
210	40	3	891035994
224	29	3	888104457
303	785	3	879485318
122	387	5	879270459
194	274	2	879539794

图 4.2 u.data 部分数据

Fig. 4.2 U.ata partial data

(2) 数据建模

MDCF 算法的第二步就是数据建模，图 4.3 为该算法部分代码，图 4.4 为代码的参数说明。

```
@classmethod
def train(
    cls,
    ratings: Union[RDD[Rating], RDD[Tuple[int, int, float]]],
    rank: int,
    iterations: int = 5,
    lambda_: float = 0.01,
    blocks: int = -1,
    nonnegative: bool = False,
    seed: Optional[int] = None,
) -> MatrixFactorizationModel:
    """..."""
    model = callMLlibFunc(
        "trainALSModel",
        cls._prepare(ratings),
        rank,
        iterations,
        lambda_,
        blocks,
        nonnegative,
        seed,
    )
    return MatrixFactorizationModel(model)
```

图 4.3MDCF 算法代码程序示例

Fig. 4.3 MDCF algorithm code program example

```

ratings : :py:class:`pyspark.RDD`
    包含评分数据的RDD, 元素可以是 Rating 类实例
    或形如 (用户ID, 商品ID, 评分) 的元组。
rank : int
    特征数, 即矩阵分解的秩。
iterations : int, 可选
    ALS算法的迭代次数, 默认为5次。
lambda\_ : float, 可选
    正则化参数, 用于防止过拟合, 默认为0.01。
blocks : int, 可选
    并行计算时使用的块的数量, 若设置为-1, 则自动配置块数量。
nonnegative : bool, 可选
    若设置为True, 则在求解过程中对矩阵分解结果施加非负约束。
seed : int 或 None, 可选
    随机种子, 用于初始化矩阵分解模型, 若设置为None,
    则使用当前系统时间作为随机种子。
"""

```

图 4.4 MDCF 算法代码程序示例参数说明

Fig. 4.4 MDCF algorithm code program example parameter description

导入所需库, 创建一个类 MyRecommend2, 用于实现推荐系统功能训练模型并计算验证集上的 RMSE

(3) 评价指标

下图所示为均方根误差:

1. 方差 $s = [(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2] / n$ ($\bar{x}$ 为平均数)。
2. 标准差 = 方差的算术平方根 (均方根误差, 亦称标准误差)。

标准差也被称为标准偏差, 或者实验标准差。

简单来说, 标准差是一组数值自平均值分散开来的程度的一种测量观念。一个较大的标准差, 代表大部分的数值和其平均值之间差异较大; 一个较小的标准差, 代表这些数值较接近平均值。

(4) 实验结果和分析

部分运行结果如下所示:

用户: 943 电影: 1682 评论: 100000

训练集: 60024 校验集: 20435 测试集: 19541

RMSE(校验集) = 8.417688515355337E-5, rank = 10, lambda = 0.01, numIter = 8

RMSE(校验集) = 2.917998259701504E-5, rank = 10, lambda = 0.01, numIter = 9

RMSE(校验集) = 8.323946508710772E-6, rank = 10, lambda = 0.01, numIter = 10

RMSE(校验集) = 7.4969577978381565E-6, rank = 11, lambda = 0.01, numIter = 8

RMSE(校验集) = 1.1810700856961077E-4, rank = 11, lambda = 0.01, numIter = 9
 RMSE(校验集) = 1.361099514896401E-5, rank = 11, lambda = 0.01, numIter = 10
 RMSE(校验集) = 2.840908514374315E-5, rank = 12, lambda = 0.01, numIter = 8
 RMSE(校验集) = 9.201099065515506E-6, rank = 12, lambda = 0.01, numIter = 9
 RMSE(校验集) = 4.040023714958279E-6, rank = 12, lambda = 0.01, numIter = 10
 测试数据集在最佳训练模型 rank = 12, lambda = 0.01, numIter = 10, RMSE = 1.3694845077975295E-5

最佳训练模型的准确率提升了: 82.88%.

789->1368: 9.099631689778658

789->700: 8.586832177103725

789->1643: 8.45173711475146

789->1019: 7.973946881975409

789->1140: 7.967043445073626

789->1131: 7.790589020780762

789->1001: 7.62642680995112

789->1128: 7.297716582625477

789->320: 7.247839465063294

789->1380: 7.239554574152051

从运行效果来看:

总共有 943 个用户, 1682 个电影 (已经去重), 100000 条评分数据; 如程序, 我们把所有数据分为三部分: 60 模型在不同参数下的均方根误差 (RMSE) 值, 以及对应的参数, 最优的参数选择均方根误差 (RMSE) 最小的参数值—即最优参数模型建立; 使用 20% 的测试模型数据来测试模型的好坏, 也就是均方根误差 (RMSE), 在最优参数模型基础上提升了 82.88% 的准确率。

本文改进算法所涉及的参数有 lambda、numIter, 实验结果分析了参数的影响, 以及不同算法之间的对比, 主要采用准确率(Accuracy)作为评价指标, 公式如下 4.3 所示。实验结果分析了参数的影响、优化的聚类算法效果以及不同算法之间的对比。

$$accuracy = \sum_{i=1}^m \frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i} \times p_i \quad (4.3)$$

其中, TP_i 和 TN_i 均代表分类正确的样本个数。 TP_i 的含义为分类结果与实际结果均为正样本。 TN_i 的含义为分类结果与实际结果均为负样本。 FP_i 和 FN_i 均代表分类错误的样本个数。 FP_i 的含义为实际结果为负样本, 但分类结果为正样本; FN_i 的含义为实际结果为正样本, 但分类结果为负样本。准确率代表所有分类正确的样本占总样本的比例。

1. 潜在因子数量 λ 值的影响

λ 值代表了模型中潜在因子 (latent factors) 的数量, 潜在因子的数量决定了模型能够捕获的用户偏好和物品特征的复杂度。从图 4.5 中可以看出, 在 λ 值较小的范围内 (例如 0.01 附近), 准确率提升百分比比较低, 这可能是因为模型过于复杂, 容易发生过拟合。当 λ 值逐渐增大 (例如 0.03 附近), 准确率提升百分比开始显著提高, 这表明增加正则化强度有助于减少过拟合, 提高了模型在验证集上的泛化能力。继续增大 λ 值 (例如 0.06 附近), 准确率提升百分比达到峰值, 说明在这个 λ 值下模型找到了较好的泛化平衡点。当 λ 值继续增大 (例如 0.09 附近), 准确率提升百分比开始下降, 这可能是因为过大的正则化强度导致模型过于简单, 出现了欠拟合现象, 无法有效捕捉数据中的关键特征。根据图像, λ 值在 0.03 到 0.06 之间的区域是模型准确率提升最为显著的区间, 其中 0.06 时准确率提升百分比达到峰值。

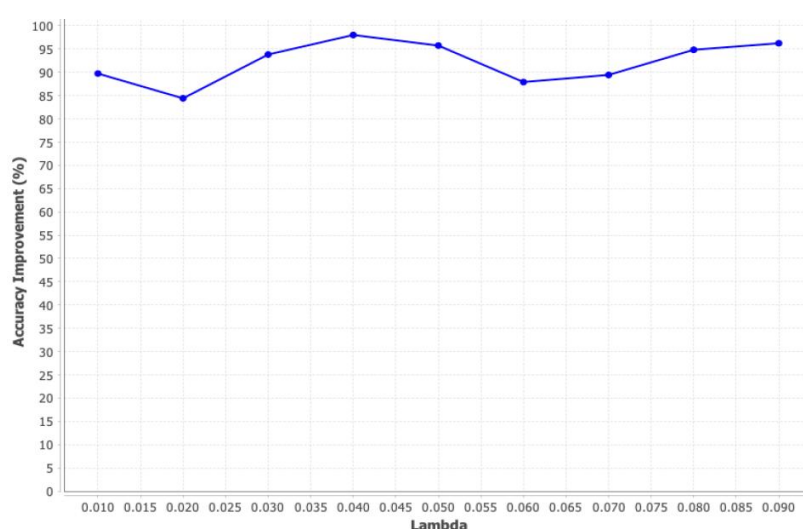


图 4.5 潜在因子数量 λ 值的影响

Fig. 4.5 The influence of λ values on the number of potential factors

2. 算法迭代次数 numIter 值的影响

numIter 代表了算法迭代次数, 迭代次数决定了算法收敛或接近最优解的程度。从图 4.6 中可以看出, 当使用较大的 NumIters 数组时 (即 NumIters:[8, 9, 10, 11, 12, 13, 14, 15]), 蓝色折线在开始时波动稍大, 然而随着迭代次数的增加, 蓝色折线逐渐趋于平缓, 表明更多的迭代次数有助于更快地提高模型性能, 在达到一定程度后, 继续增加迭代次数对模型性能的提升效果有限。适当的选择迭代次数有助于避免过度训练, 保持模型在训练集和测试集上的表现一致性。

相反，当使用较小的 NumIters 数组时（即 NumIters:[1, 1, 1, 1, 1, 1, 1, 1]），红色折线在整个迭代过程的变化起伏相较于蓝色折线也更大，表明使用较少的迭代次数，在一定程度上会降低模型的准确度。

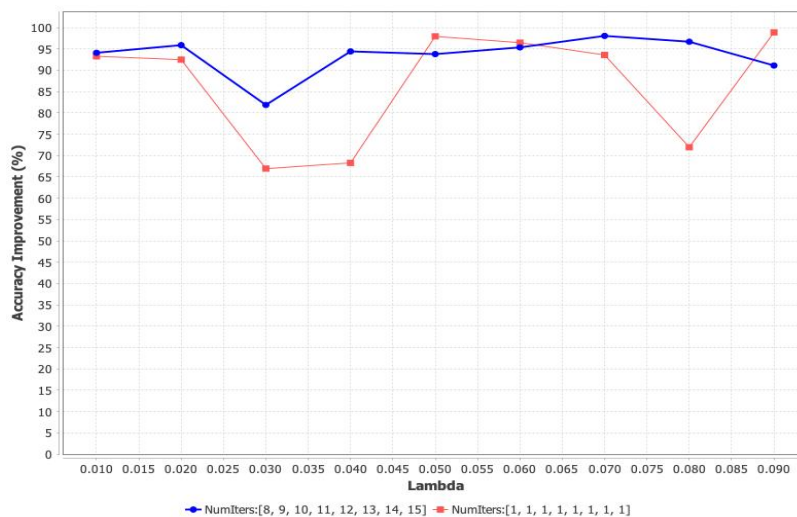


图 4.6 算法迭代次数 numIter 值的影响

Fig. 4.6 The numIter value of the number of algorithm iterations

本章小结

本章主要介绍了基于矩阵分解的协同过滤推荐算法-MDCF 及其相较于传统协同过滤算法的改进之处。MDCF 算法利用 Spark 的分布式计算能力提高了协同过滤在大规模数据上的计算效率和扩展性，不仅能够处理更大规模的用户-物品交互数据，而且通过矩阵分解，增强了推荐的准确性和模型优化。

同时，MDCF 算法支持隐式反馈数据，能够根据用户的隐式行为，如点击、浏览等，来进行个性化推荐。

在算法实现层面，重点探讨了 MDCF 算法的具体实现，特别是该算法如何使用交替最小二乘法（ALS）进行矩阵分解。在推荐系统中，评分矩阵通过将用户和物品映射到低维隐向量空间中进行矩阵分解，以此来预测缺失的评分值，从而实现推荐。MDCF 算法通过交替优化用户和物品的隐向量矩阵，最终达到对评分矩阵的有效逼近，解决了大规模稀疏矩阵补全的问题。

实验过程中，选择了 MovieLens 数据集（ml-100k 版本）作为研究对象，其中包括 943 个用户对 1682 部电影的 100000 条评分数据。实验步骤首先展示了数据预处理阶段，随后详细描述了如何构建协同过滤模型，包括参数设置（如特征数量、迭代次数、正则化参数等）和数据适配过程。实验中还涵盖了训练集、验证集和测试集的划分，以及超参数搜索策略，以期找到最佳的模型参数组合，进而获得最低的验证集均方根误差（RMSE），提升推荐系统的性能表现。

通过在实际数据集上进行实验验证,我们发现本章所提出的 MDCF 算法在处理矩阵稀疏性方面表现出色。具体来说,与传统的协同过滤算法相比,该算法能够更准确地预测用户对未知物品的评分,并且能够在评分矩阵非常稀疏的情况下仍然保持较高的推荐准确率。

第五章 融合多层感知机的协同过滤推荐算法

5.1 引言

本章针对传统的协同过滤算法存在的冷启动问题，提出了融合多层感知机的协同过滤改进算法-MPCF,采用神经网络的嵌入层用作特征表示、多层非线性变换学习复杂关系以及模型可视化等手段。实验结果显示 MPCF 算法不仅在冷启动问题等方面相较于传统的基于用户协同过滤算法具有优越性，而且在捕捉复杂关系、预测准确率、灵活性和可扩展性都有不错表现。

本章提出的算法中的多层感知机又叫人工神经网络，神经网络可以通过将用户和物品的特征向量作为输入，来预测用户对新物品的评分，从而解决冷启动问题。对数据稀疏性的鲁棒性：它可以自动学习用户和物品的特征向量，并能够通过这些向量来预测评分，因此对于数据稀疏的情况也能进行有效的预测。

更好的特征表示能力：传统的协同过滤算法通常将用户和物品表示为简单的 ID 或向量，这些表示可能无法充分反映用户和物品的特性。本章所提出的 MPCF 算法利用神经网络的嵌入层，将用户和物品映射到低维稠密向量空间，这些向量能够捕获用户和物品的深层特征。这种表示方式不仅更加紧凑，而且能够保留更多的信息，有助于提高推荐的准确性。

非线性建模能力：传统的协同过滤算法，如基于用户的协同过滤，主要依赖于线性模型来捕捉用户和物品之间的关系。这种线性模型在处理复杂、非线性的用户-物品交互时可能受到限制。MPCF 算法通过引入神经网络，通过多层非线性变换来学习用户和物品之间的复杂关系。这使得 MPCF 算法能够更准确地捕捉用户兴趣，并为用户生成更精准的推荐。

可解释性和灵活性：MPCF 算法可以通过调整网络结构和参数来优化预测准确率，并且可以通过可视化方法来解释预测结果。

5.2 神经网络算法分类

MPCF 算法利用多层感知机，而多层感知机属于神经网络的范畴，神经网络算法的分类是极其丰富多样的，每一种类型都有其独特的工作原理和应用场景。下面是一些主要的神经网络算法分类及其详细介绍，并附带了相关的数学公式：

(1) 前馈神经网络：

①多层感知机 (MLP)：多层感知机也称人工神经网络，其输出是输入的加权和经过非线性激活函数后的结果。对于单个神经元，其输出可以表示为：

$$y = \sigma(\sum_i w_i x_i + b) \quad (5.1)$$

其中, x_i 是输入, w_i 是权重, b 是偏置项, σ 是激活函数 (如 Sigmoid、ReLU 等)。

②卷积神经网络 (CNN) :

CNN 通过卷积层对输入图像进行局部特征提取。卷积操作可以表示为:

$$S(i, j) = (I * K)(i, j) = \sum_{m, n} I(i - m, j - n) K(m, n) \quad (5.2)$$

其中, I 是输入图像, K 是卷积核, S 是卷积后的特征图。

(2) 循环神经网络

①简单循环神经网络 (RNN) :

RNN 通过引入循环连接来捕捉序列数据中的时间依赖性。对于时刻 t 的隐藏状态 h_t , 其更新公式为:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (5.3)$$

其中, x_t 是当前时刻的输入, h_{t-1} 是上一时刻的隐藏状态, W_{xh} 和 W_{hh} 是权重矩阵, b_h 是偏置项。

②长短时记忆网络 (LSTM) :

LSTM 通过引入门控机制和记忆单元来解决 RNN 的梯度消失问题。其核心部分包括遗忘门 f_t 、输入门 i_t 、输出门 o_t 和记忆单元 c_t , 其更新公式如下

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{c}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned} \quad (5.4)$$

其中, σ 是 Sigmoid 函数, $\odot$ 表示逐元素相乘, W 和 b 是相应的权重和偏置项。

(3) 自编码器

自编码器通常通过最小化重构误差来学习数据的内在表示。其基本结构包括编码器和解码器两部分。编码器的输出是输入数据的低维表示, 而解码器则试图从该表示中重构原始输入。重构误差通常使用均方误差 (MSE) 或交叉熵损失来衡量。

(4) 生成对抗网络 (GAN)

GAN 由生成器和判别器组成。生成器的目标是生成尽可能接近真实数据的假数据, 而判别器的目标是区分真实数据和假数据。它们的优化过程是一个极小极大博弈问题, 可以表示为:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [1 - D(G(z))] \quad (5.5)$$

其中, G 是生成器, D 是判别器, $p_{data}(x)$ 是真实数据的分布, $p_z(z)$ 是生成器输入噪声的分布。

(5) 其他类型的神经网络

①自组织映射网络 (Self-Organizing Maps, SOM) 自组织映射网络是一种无监督学习算法, 用于将高维数据映射到低维空间中。其基本原理是通过竞争学习和自组织的方式, 将输入数据投影到一个二维或三维的拓扑结构上, 以便保持数据之间的拓扑关系。SOM 包含一个权重矩阵, 每个节点对应于该矩阵中的一个位置, 并且与输入空间中的样本之间存在拓扑关系。SOM 的学习过程分为两个阶段: 竞争和合作。在竞争阶段, 与输入模式最相似的节点获胜并更新其权重; 在合作阶段, 获胜节点周围的邻居节点也更新其权重以更好地拟合输入模式。SOM 的权重更新规则可以表示为:

$$\Delta W_{ji} = \eta(t) \cdot h_{ji}(t) \cdot (X(t) - W_{ji}(t)) \quad (5.6)$$

其中, ΔW_{ji} 是节点 j 的权重向量 W_{ji} 的变化量, $\eta(t)$ 是学习率, $h_{ji}(t)$ 是邻域函数, $x(t)$ 是当前输入模式, w_{ji} 是节点 j 的权重向量。

②Hopfield 网络

Hopfield 网络是一种反馈型神经网络, 用于模式识别和联想记忆^[41]。它由一组神经元组成, 这些神经元之间的连接是全连接的, 并且是对称的。Hopfield 网络的更新规则如下所示:

$$S_i(t+1) = \text{sign}(\sum_{j=1}^N W_{ij} S_j(t)) \quad (5.7)$$

其中, $s_i(t)$ 表示神经元 i 在时间 t 的状态, w_{ij} 是连接权重。

③学习矢量量化 (Learning Vector Quantization, LVQ) 网络

LVQ 网络是一种监督学习算法, 用于模式分类。它通过向量量化将输入数据映射到预定义的类别, 是一种简单而有效的分类器。LVQ 网络的学习规则如下:

$$W_i(t+1) = W_i(t) + \Delta W_i \quad (5.8)$$

其中, w_i 是向量量化的权重向量, Δw_i 是根据输入模式和预定义类别计算出的更新量。

④Kohonen 网络

Kohonen 网络是一种竞争型神经网络, 用于聚类和数据可视化。它通过竞争学习机制自组织形成拓扑结构, 可用于发现数据中的潜在模式和分布。Kohonen 网络的权重更新规则如下:

$$W_{ij}(t+1) = W_{ij}(t) + \eta(t) \cdot h_{ij}(t) \cdot (X(t) - W_{ij}(t)) \quad (5.9)$$

其中, w_{ij} 是神经元 i 的权重向量, $\eta(t)$ 是学习率, $h_{ij}(t)$ 是邻域函数, $x(t)$ 是当前输入模式。

这些不同类型的神经网络在多个领域都有广泛的应用，它们都有其独特的优势和适用场景。在实际应用中，需要根据具体问题和需求来选择合适的算法，并结合数据集的特性进行参数调整和模型优化。同时，随着深度学习技术的不断发展，新的神经网络算法不断涌现。这些算法在结构、优化方法、应用领域等方面不断创新，为人工智能领域的发展注入了新的活力。同时，随着计算能力的提升和数据资源的丰富，神经网络算法的性能和应用范围也在不断扩展。

5.3 MPCF 算法

本章通过融合多层感知机来对基于用户协同过滤的推荐算法进行改进。MPCF 首先利用神经网络对用户和物品进行向量化表示，并通过多层感知机实现了非线性映射，从而能够更好地捕捉用户和物品之间的复杂关系。MPCF 还通过引入负样本来训练模型，提高了推荐系统的多样性。

下图所示为 MPCF 框架结构：

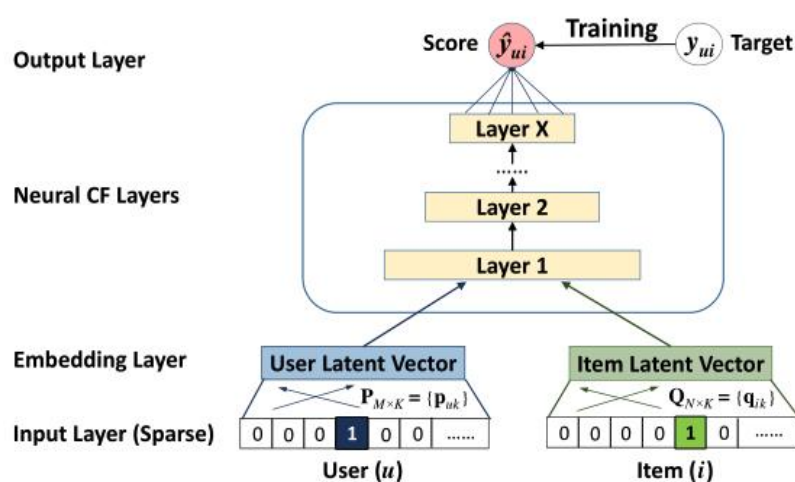


图 5.1 MPCF 框架

Fig. 5.1 MPCF framework

5.3.1 算法描述

尽管目前很多推荐系统已经使用了深度学习，但是他们主要是用它（深度学习）来对一些辅助信息（auxiliary information）建模，比如描述文字的项目和音乐的声学特征。当涉及到建模协同过滤的关键因素（key factor）——用户和项目（item）特征之间的交互的时候，他们仍然采用矩阵分解的方式，并将内积（inner product）做为用户和项目的潜在特征点乘。而我们通过使用神经结构代替内积，这可以达到从数据中学习任意函数的效果。

MPCF 算法的核心思想是利用神经网络来模拟用户和物品之间的交互过程，从而预测用户对物品的评分或偏好。下面，我们将用数学公式来详细解释 MPCF 的工作原理。

首先，MPCF 将用户和物品嵌入到低维的向量空间中，即每个用户和物品都由一个向量表示。假设用户 u 的嵌入向量为 $p_u \in R^K$ ，物品 i 的嵌入向量为 $q_i \in R^K$ ，其中 K 是嵌入空间的维度。

然后，MPCF 使用多层感知机（MLP）来学习用户和物品之间的交互函数。这个交互函数可以表示为：

$$\phi(p_u, q_i) = h^T f(W^T \begin{bmatrix} p_u \\ q_i \end{bmatrix} + b) \quad (5.10)$$

其中， $W \in R^{H \times (2K)}$ 和 $b \in R^H$ 是 MLP 第一层的权重和偏置， H 是隐藏层的维度； f 是非线性激活函数； $h \in R^H$ 是 MLP 输出层的权重； $\phi(p_u, q_i)$ 是用户 u 对物品 i 的预测评分。

在训练过程中，MPCF 使用均方误差（MSE）作为损失函数，并引入负样本来增加模型的泛化能力。负样本是指用户实际没有交互过的物品。对于每个正样本（即用户实际交互过的物品），MPCF 会随机选择若干个负样本。

损失函数的公式如下：

$$L = \sum_{(u,i) \in Y} w_{ui} (r_{ui} - \phi(p_u, q_i))^2 + \lambda (\|P\|_F^2 + \|Q\|_F^2 + \|W\|_F^2 + \|h\|_2^2 + \|b\|_2^2) \quad (5.11)$$

其中， Y 是所有用户-物品交互的集合； r_{ui} 是用户 u 对物品 i 的实际评分； w_{ui} 是样本权重；

λ 是正则化系数； $\|\cdot\|_F^2$ 是 Frobenius 范数，用于约束权重矩阵； $\|\cdot\|_2^2$ 是 L2 范数，用于约束向量。

最后，MPCF 使用梯度下降等优化算法来最小化损失函数，从而学习到用户和物品的嵌入向量以及 MLP 的参数。在预测阶段，对于任意给定的用户 u 和物品 i ，MPCF 可以直接使用学到的嵌入向量和 MLP 参数来计算预测评分 $\phi(p_u, q_i)$ ，并根据评分进行推荐。

MPCF 算法利用多层感知机对用户和物品进行深度特征提取，其复杂度分析如下：

时间复杂度：MPCF 算法的时间复杂度受神经网络层数、每层神经元数量以及训练样本数量的影响。设神经网络层数为 L ，每层神经元平均数量为 N ，训练样本数量为 S ，则 MPCF 算法的时间复杂度大致为 $O(LSN^2)$ 。

空间复杂度：MPCF 算法需要存储神经网络的权重、偏置项以及中间层的激活值。因此，其空间复杂度与神经网络的结构和规模紧密相关，大致为 $O(LN^2)$ 。

下图 5.2 所示为本文对传统协同过滤算法进行改进的 MPCF 算法流程图：

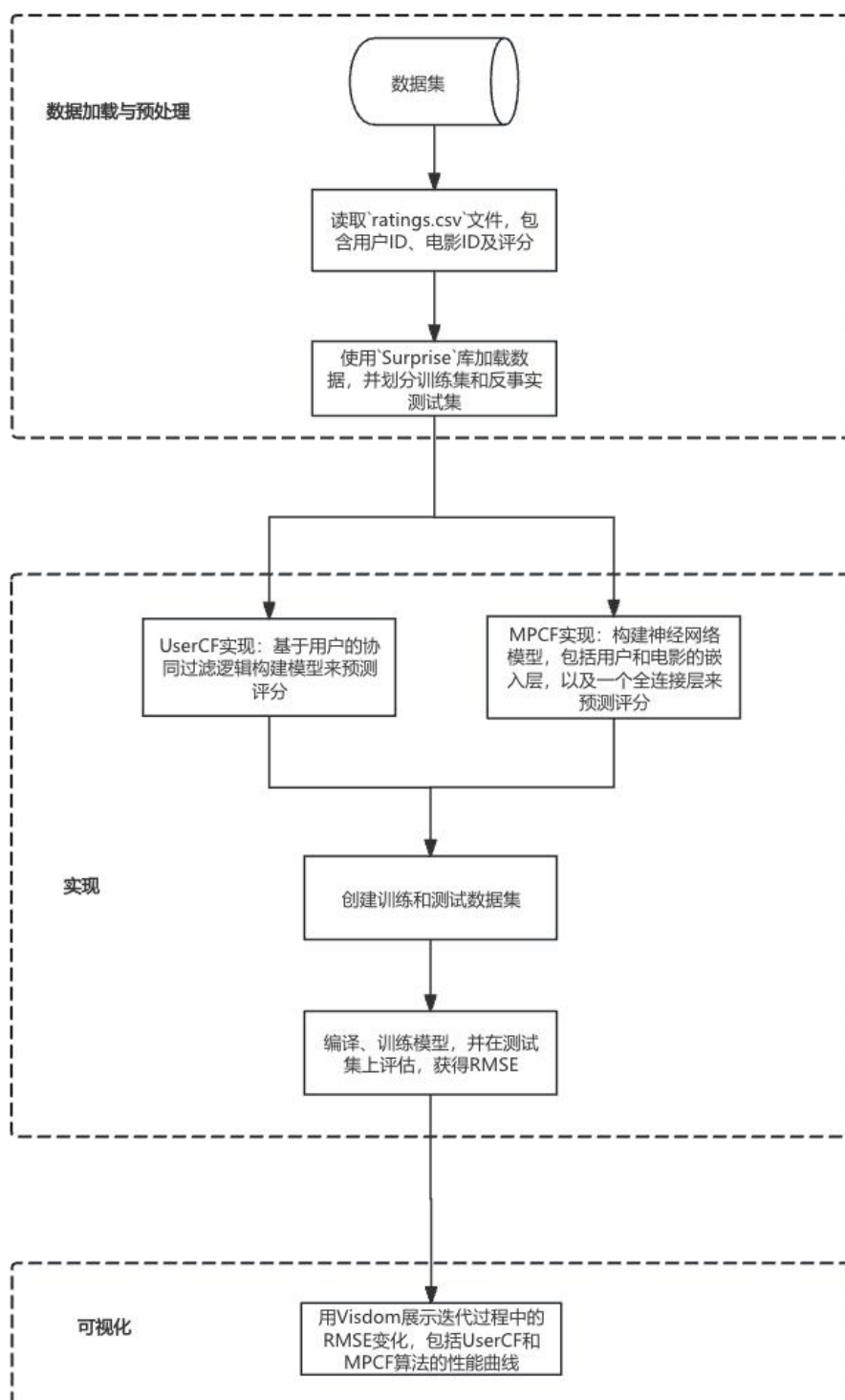


图 5.2 MPCF 算法流程图

Fig. 5.2 Flowchart of MPCF algorithm

5.3.2 实验过程及分析

本次实验采用的数据集依旧是 movielens 上的数据集, 我们将使用 (100k) 版本作为我们的数据集, 该数据集包含了 943 个用户对 1682 部电影的评分^[42], 评分范围是 1-

5, 我们使用 随机划分法, 采用随机抽样的方式将 80% 的数据作为训练集, 10% 的数据作为验证集, 10% 的数据作为测试集。

(1) 数据集转换

我们首先需要将数据集转换为 PyTorch 可以处理的格式。我们定义一个名为 MovieLens- Dataset 的类, 该类继承自 torch.utils.data.Dataset, 并覆盖 init、len 和 getitem 方法。

在 init 方法中, 我们加载数据集文件, 并计算用户和电影的数量。然后, 我们创建两个字典, 分别将用户和电影的 ID 映射为连续的整数。最后, 我们将数据集中的每一行作为一个元组存储在 self.data 中。

在 len 方法中, 我们返回数据集中元组的数量。

(2) 模型构建

接下来, 我们将介绍如何构建 MPCF 算法模型。MPCF 算法模型将用户和电影的特征表示作为输入, 并输出一个预测的评分值。我们将使用两个神经网络来生成用户和电影的特征表示。这两个网络共享相同的结构, 由一些全连接层和激活函数组成。最后, 我们将这两个特征表示向量相乘, 并通过一些全连接层和激活函数输出预测的评分值。

我们定义一个名为 NeuralCollaborativeFiltering 的类, 该类继承 torch.nn.Module, 并实现 init 和 forward 方法。

在 init 方法中, 我们定义了模型中所需的层, 包括两个嵌入层、多个全连接层和一个输出层。其中, 嵌入层将用户 ID 和电影 ID 映射为特征表示向量, 全连接层和激活函数将特征表示向量转换为更高维的特征表示, 输出层将这些特征表示向量相乘并输出预测的评分值。

在 forward 方法中, 我们首先通过嵌入层获取用户和电影的特征表示向量, 并将它们拼接在一起。然后, 我们通过全连接层和激活函数将特征表示向量转换为更高维的特征表示, 并通过输出层将这些特征表示向量相乘并输出预测的评分值。

(3) 评级指标

现在我们已经定义了数据集和模型, 接下来是训练模型的步骤。本实验使用均方根误差 (RMSE) 作为评估指标, 并使用随机梯度下降 (SGD) 优化器来更新模型的参数。

均方根误差(RMSE): 代表预测值和真实值之间的平方误差平均值的平方根, 相较于平均绝对误差 (MAE) 对异常值更为敏感。RMSE 的取值范围是 $[0, +\infty)$, 数值越小表示模型的预测误差越小, 模型的预测能力越强。公式如下:

$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1} (\hat{y}_t - y_t)^2} \quad (5.12)$$

其中: n 代表样本数; $\hat{y}_t$ 为预测值; y_t 为真实值;

(4) 训练模型

我们定义一个名为 `train` 的函数, 该函数将负责训练模型。我们将使用交叉熵损失函数来计算模型的损失。该函数接受三个参数: 训练集、验证集和模型。

在训练函数中, 我们首先将训练集和验证集转换为 PyTorch 数据加载器, 并定义交叉熵损失函数和随机梯度下降优化器。在每个 `epoch` 中, 我们遍历训练集并更新模型的参数, 同时记录训练损失。然后, 我们遍历验证集并计算验证损失。如果验证损失比当前最好的验证损失更小, 我们将保存当前模型的参数。最后, 我们输出当前 `epoch` 的训练损失和验证损失。

(5) 实验结果和分析

输出结果如下:

```
1 Epoch 1/10, Train Loss: 1.3809, Val Loss: 1.1251
2 Epoch 2/10, Train Loss: 1.1124, Val Loss: 1.1107
3 Epoch 3/10, Train Loss: 1.1031, Val Loss: 1.1028
4 Epoch 4/10, Train Loss: 1.0981, Val Loss: 1.0987
5 Epoch 5/10, Train Loss: 1.0947, Val Loss: 1.0963
6 Epoch 6/10, Train Loss: 1.0924, Val Loss: 1.0946
7 Epoch 7/10, Train Loss: 1.0911, Val Loss: 1.0916
8 Epoch 8/10, Train Loss: 1.0900, Val Loss: 1.0905
9 Epoch 9/10, Train Loss: 1.0893, Val Loss: 1.0893
10 Epoch 10/10, Train Loss: 1.0889, Val Loss: 1.0889
```

在训练完成后, 我们需要测试模型的性能。我们定义一个名为 `test` 的函数, 该函数接受测试集和模型作为参数, 并计算模型在测试集上的 RMSE。

在测试函数中, 我们首先将测试集转换为 PyTorch 数据加载器, 并定义 MSE 损失函数。然后, 我们遍历测试集并计算模型在测试集上的损失。最后, 我们计算 RMSE 并输出结果。

输出结果如下:

```
TestRMSE:1.0431
```

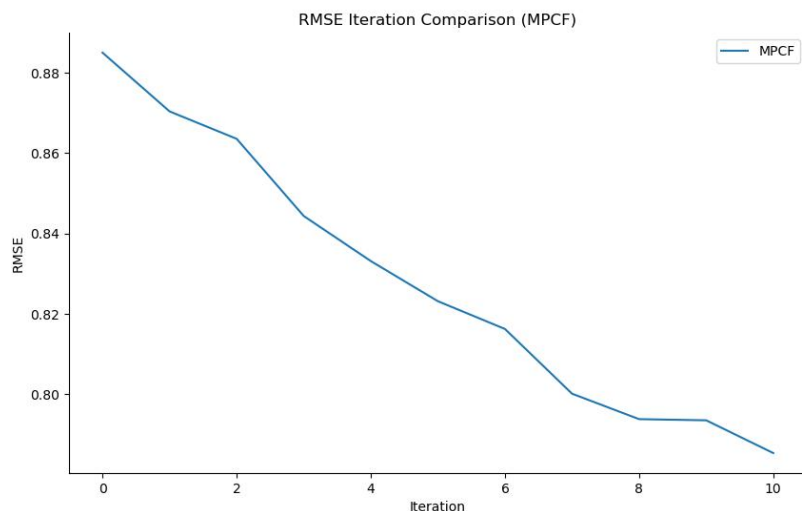



图 5.3 不同迭代次数对 MPCF 算法预测精度的影响

Fig. 5.3 Influence of different iteration times on prediction accuracy of MPCF algorithm

图 5.3 中显示的是 MPCF 算法在不同迭代次数下的均方根误差 (RMSE) 的变化情况。图中的横轴表示迭代次数, 纵轴表示 RMSE。可以看到, 随着迭代次数的增加, MPCF 模型的 RMSE 逐渐减小, 说明模型在不断优化, 预测精度在提高。

图 5.4 中比较两种不同推荐算法——UserCF 和 MPCF 的均方根误差 (RMSE) 与迭代次数之间的关系。

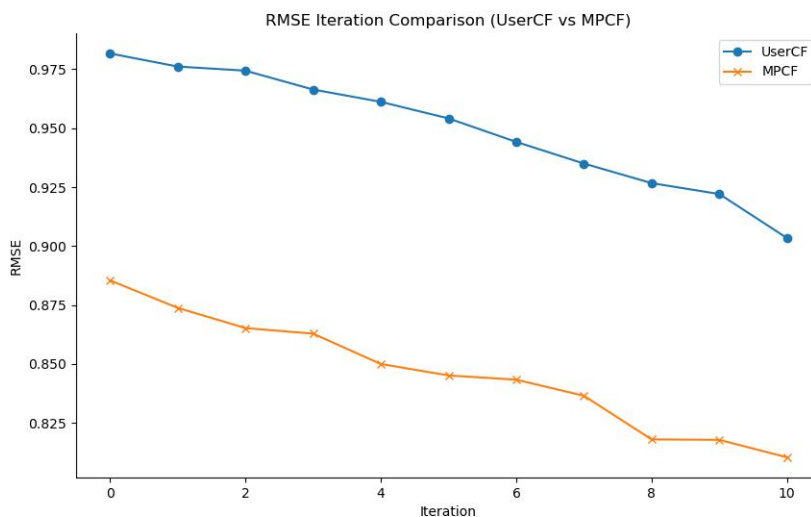


图 5.4 不同算法的预测精度比较

Fig. 5.4 Comparison of prediction accuracy of different algorithms

在图 5.4 中，横轴代表迭代次数，纵轴代表 RMSE 值，数值越小表示模型的预测误差越小，即模型的表现越好。

蓝色折线表示 UserCF 算法在不同迭代次数下的 RMSE 值。从图中可以看出，随着迭代次数的增加，UserCF 模型的 RMSE 值呈现下降的趋势，这说明模型的预测误差在逐渐减小，预测结果的准确度在逐渐提升。橙色折线表示 MPCF 算法在不同迭代次数下的 RMSE 值。从图中可以看出，MPCF 模型的 RMSE 值随着迭代次数的增加也呈现下降的趋势。然而，UserCF 的 RMSE 值下降速度明显快于 MPCF，这意味着 UserCF 在迭代过程中能够更快地收敛到较低的误差水平。因此，在迭代次数相同的情况下，UserCF 的预测精度通常会比 MPCF 更高。

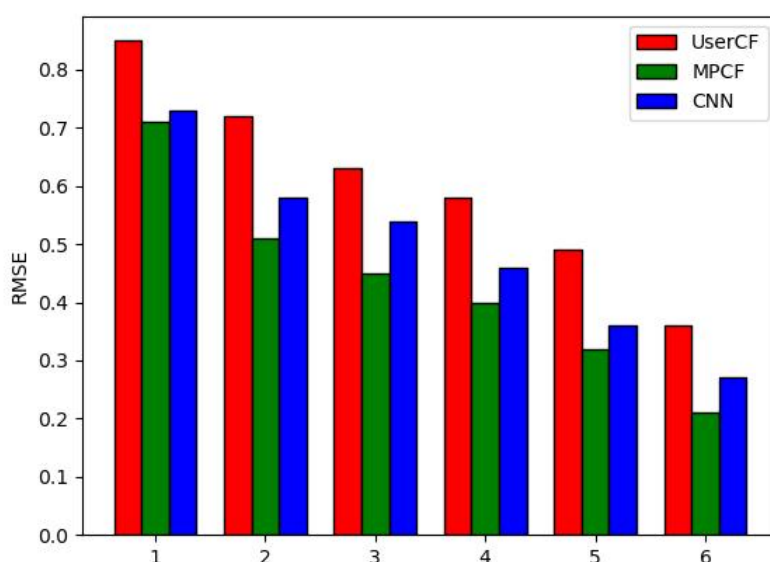


图 5.5 算法准确性对比

Fig. 5.5 Algorithm accuracy comparison

为了验证算法在不同领域的适用性及效果，我们还使用了 PixelRec 数据集进行试验，该数据是一个大规模的短视频平台下图像推荐数据集，目前公开了 8 万原始图文数据和 5 万用户的交互数据，该数据集可以极大的帮助推动通过直接使用原始像素来建模物品的推荐模型的研究。

在 PixelRec 数据集上，为了考察算法的准确率，我们把该算法的试验结果和 UserCF（基于用户的协同过滤）算法和在 CNN（卷积神经网络）算法进行实验对比。CNN 主要用于图像处理，但在推荐系统中也被用于提取文本和图像等非结构化数据的特征。例如，在基于图像的推荐系统中，CNN 可以提取图像的视觉特征，并结合用户的历史行为数据进行推荐。我们采用随机抽样的方式将 80% 的数据作为训练集，10% 的数据作为验证集，10% 的数据作为测试集。实验结果如图所示，上图 5.5 中，可以

看出随着算法复杂度的增加（从简单的 UserCF 到更复杂的 MPCF 和 CNN），预测精度也在提高。这显示了更先进的推荐算法能够更有效地捕捉用户和物品之间的关系，从而做出更准确的预测。从左至右，UserCF 的 RMSE 值是最大的，MPCF 和 CNN 的 RMSE 明显小于 UserCF 的 RMSE。这表明我们的算法的预测精度更高。

本章小结

在本文中，我们深入探讨了多种神经网络算法在分类任务中的应用及其优势。特别地，我们提出了自己的 MPCF 算法，并对其进行了详细的解析与实现。

第四章提出的 MDCF 算法主要侧重于解决协同过滤中的矩阵稀疏性问题，而本章提出的 MPCF 算法则主要解决冷启动问题。两者都采用了降维技术，但具体实现方式不同。MDCF 通过矩阵分解实现，而 MPCF 则通过神经网络的嵌入层实现。这种技术上的互补使得两者在解决不同问题时能够发挥各自的优势。

本章提出的 MPCF 算法融合了多层感知机，结合了深度学习的强大表征学习能力和协同过滤的个性化推荐能力，从而能够更精准地捕捉用户和物品之间的复杂关系。为了验证其有效性，我们在 MovieLens 数据集和 PixelRec 数据集上进行了模型的训练和测试。在此过程中，我们不仅对模型架构进行了精心设计，还仔细选择了合适的优化算法和参数，以确保模型能够达到最佳性能。

在选择评估指标时，我们使用了均方根误差（RMSE）作为本实验的评估指标，并使用随机梯度下降（SGD）优化器来更新模型的参数。

通过一系列的实验和比较，我们发现 MPCF 算法在推荐系统任务中表现出了显著的优势。通过与其他推荐算法相比，MPCF 算法的 RMSE 值更优。这一结果不仅证明了 MPCF 算法的有效性，也为我们未来在推荐系统领域的研究提供了有力的支持。

展望未来，我们将继续探索利用神经网络对推荐算法进行改进，并尝试将其应用于更广泛的推荐场景中。同时，我们也将关注其他新兴的深度学习算法和技术，以期在推荐系统领域取得更多的突破和进展。

第六章 系统设计与实现

6.1 系统功能结构设计

本章主要是设计开发一个电影推荐系统。构建高效的推荐系统是一项综合性工程，旨在通过精准定制的内容满足用户的特定需求。该系统的架构围绕几个核心环节展开：首先，借助数据抓取技术，从各类电影相关信息网站获取电影数据和用户数据，为推荐引擎提供丰富原始数据。随后，将用户数据和电影信息数据分别存储在不同的数据集内，以便于后续对数据进行深入分析。

核心的推荐处理模块在此基础上运作，运用协同过滤算法等多种策略，从而生成高度个性化的推荐清单。然后设计出图形用户界面(GUI)进行交互，作为用户与系统间的桥梁，不仅让用户能轻松浏览和搜索想看的电影信息，还收集用户反馈，持续优化推荐算法与用户体验。

为了便于管理和使用，本电影推荐系统主要分为用户管理和电影管理两大模块，系统总体设计结构图如下图 6.1 所示。

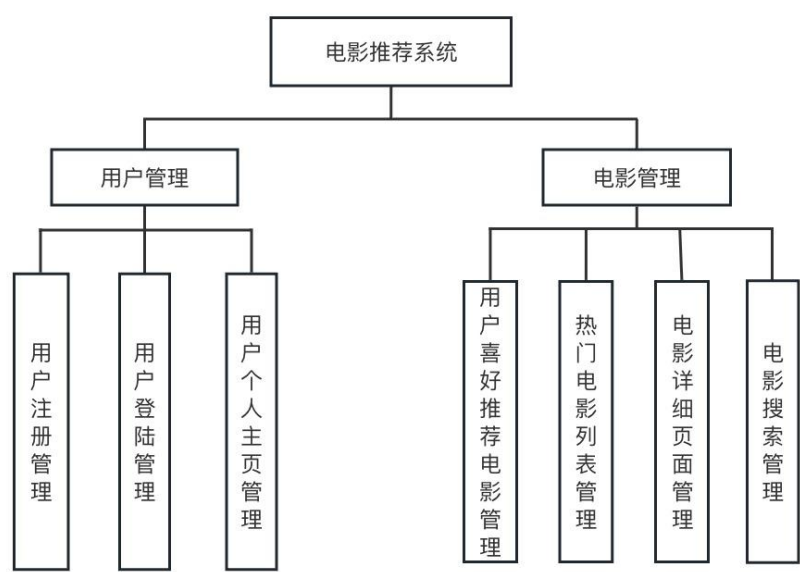


图 6.1 系统总体结构设计图

Fig. 6.1 System overall structure design drawing

6.1.1 用户管理

用户可以进行用户名和密码的注册，然后使用其进行登陆，在进入界面后，点击个人主页，可以看到关于用户及其相应喜好类型的个人信息。

6.1.2 电影管理

用户登陆后，可以展示根据用户喜好推荐的电影列表，以及当前数据存储的所有电影列表，主要包括电影的名称、主演、类型、链接。用户可以通过电影搜索来模糊查询相应的电影信息，查询将返回匹配的所有电影的名称、评分、链接等信息，也可以通过电影详细界面来精确搜索相应的电影信息，查询将返回该电影的导演、国家、主演、评分、类型、上映日期以及电影相关简介等信息。

6.1.3 推荐算法模块

算法分析模块集成了 UBCF、MDCF 和 MPCF 等多种推荐算法，通过调用这些算法为用户提供个性化推荐。用户管理模块负责用户信息的存储和管理，电影管理模块则负责电影信息的维护和更新。推荐算法模块与用户管理和电影管理模块紧密协作，根据用户的历史行为和当前需求，实时计算并推荐合适的电影。

在 UBCF 算法实现中，系统首先收集用户对电影的评分数据或浏览记录，构建用户-物品交互矩阵。随后，通过计算用户之间的相似度，找出与目标用户相似的用户群体。

MDCF 算法通过矩阵分解技术，将用户-物品交互矩阵分解为两个低维矩阵（用户特征矩阵和物品特征矩阵），从而缓解数据稀疏性问题。在系统实现中，我们使用随机梯度下降等优化算法来迭代更新这两个矩阵，直至收敛。通过计算用户和物品在低维空间中的内积，系统可以预测用户对未评分物品的偏好。

MPCF 算法利用多层感知机（MLP）来捕获用户和物品的深层特征，提高推荐的准确度。在系统实现中，我们首先通过嵌入层将用户和物品映射到低维稠密向量空间。然后，利用 MLP 的多个隐藏层对用户和物品的向量进行非线性变换，以学习它们之间的复杂关系。最后，通过输出层预测用户对物品的偏好。

基于这些预测的偏好，系统为目标用户生成电影推荐列表。

系统通过 GUI 展示电影推荐列表，用户可以通过点击浏览推荐的电影。用户还可以对推荐结果进行评分或反馈，系统将根据用户的反馈自动调整推荐算法中的参数或重新训练模型，以提高推荐的准确度和用户满意度。

6.1.4 系统使用流程

电影推荐系统的使用流程图如图 6.2 所示。

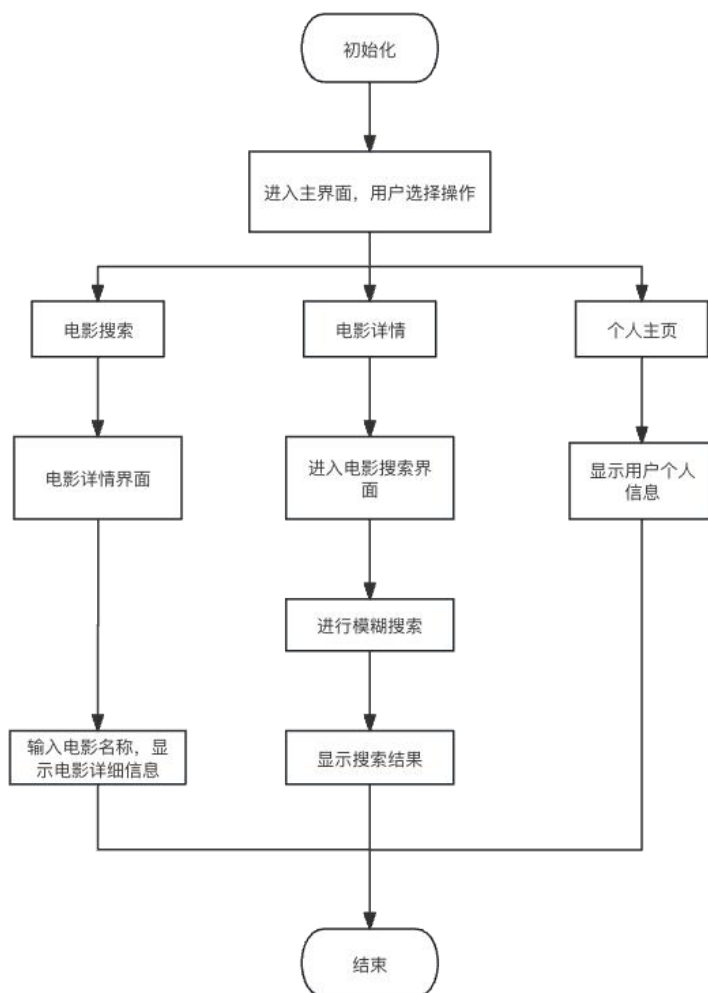


图 6.2 电影推荐系统使用流程图

Fig. 6.2 The movie recommendation system uses flowcharts

6.2 系统界面功能实现

6.2.1 登录注册界面

登录注册界面是经典的用户图形界面，在 QQ 等平台都有类似的界面，使用 2 个 QLabel、2 个 QLineEdit、2 个 QPushButton，再使用嵌套布局即可实现界面。此外通过 setPlaceholderText 设置占位字符串，再通过 setEchoMode 设置密码掩码，且连接检测用户名和密码输入的函数，使得输入用户名或密码为空时，登录按钮无法按下。然后设置注册按钮的 clicked 信号连接注册的界面。

图 6.3 所示为用户登录界面：

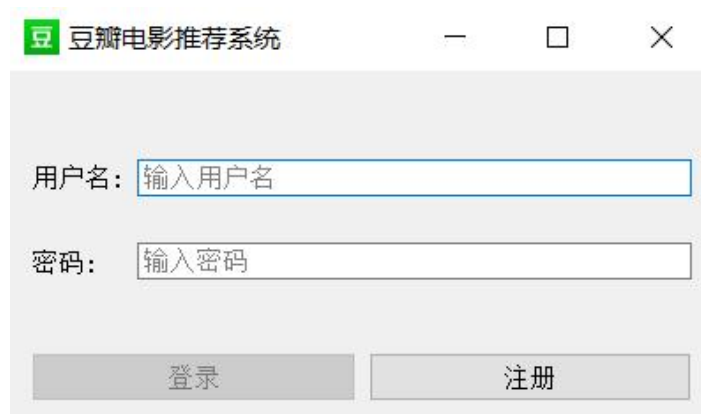


图 6.3 系统登陆界面

Fig. 6.3 System login interface

接下来是注册界面的实现，如图 6.4 所示。和登录界面类似，3 个 QLabel、3 个 QLineEdit、1 个 QPushButton，再加上嵌套布局即可实现。

该登录注册界面实现以下异常处理：

- ①用户名无法重复注册
- ②注册时两次密码输入应一致



图 6.4 系统注册界面

Fig. 6.4 System registration interface

用户使用用户名为 cabaret，密码为 123456 进行注册，注册成功后界面会显示注册成功，如图 6.5 所示。

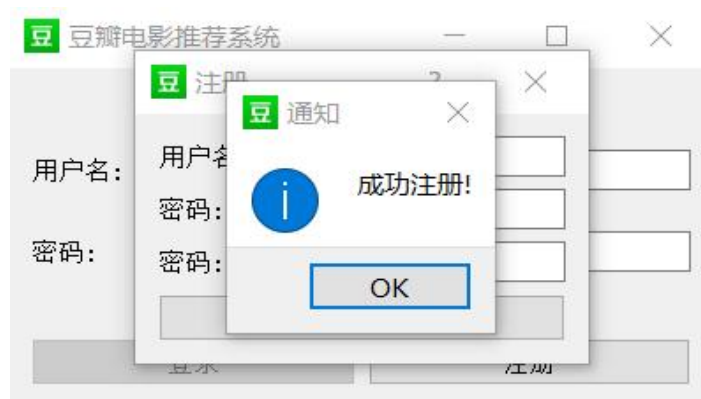


图 6.5 成功注册界面

Fig. 6.5 Successful registration interface

如图 6.6 所示，用户新注册时，会弹出一个窗口，询问用户喜欢的电影类型：此处输入“喜剧”进行测试。

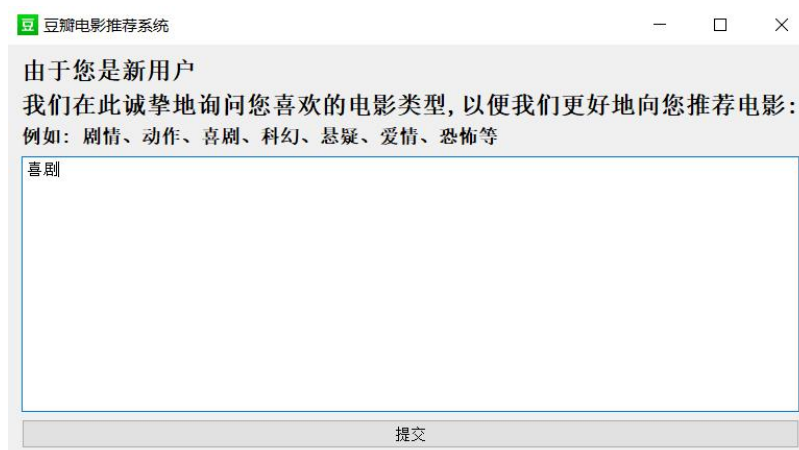


图 6.6 用户喜好填写界面

Fig. 6.6 Users prefer to fill in the interface

结果如下图 6.7 所示，可以看到，系统根据用户喜欢的电影类型给用户进行了个性化推荐。电影列表中的电影类型均为用户注册时填写的类型。

豆瓣电影推荐系统

欢迎您！

根据您的喜好为您推荐以下电影：

	电影名称	主演	电影评分	电影类型	上映时间	电影链接
1	《美人鱼》	邓超 罗志祥 ...	6.7	喜剧	2016	https://...
2	《飞驰人生》	沈腾 黄景瑜 ...	6.9	喜剧	2019	https://...
3	《超能陆战...	斯科特·安第...	8.7	喜剧	2015	https://...
4	《神偷奶爸》	史蒂夫·卡瑞...	8.6	喜剧	2010	https://...
5	《夏洛特烦...	沈腾 马丽 ...	7.6	喜剧	2015	https://...
6	《调音师》	阿尤斯曼·库...	8.3	喜剧	2019	https://...
7	《唐人街探案...	王宝强 刘昊...	6.7	喜剧	2018	https://...
8	《功夫》	周星驰 元秋 ...	8.6	喜剧	2004	https://...
9	《西红柿首富...	沈腾 宋芸桦 ...	6.6	喜剧	2018	https://...
10	《唐伯虎点秋...	周星驰 巩俐 ...	8.6	喜剧	1993	https://...
11	《大话西游之...	周星驰 吴孟...	9.0	喜剧	1995	https://...
12	《大话西游之...	周星驰 吴孟...	9.2	喜剧	1995	https://...
13	《寻梦环游...	安东尼·冈萨...	9.1	喜剧	2017	https://...
14	《疯狂动物...	金妮弗·古德...	9.2	喜剧	2016	https://...
15	《校园情圣》	刘昊然 宋... 丹...	8.1	喜剧	2020	https://...

图 6.7 电影喜好推荐列表

Fig. 6.7 Recommended list of movie preferences

6.2.2 用户主界面

登录成功后进入主界面，如图 6.8 所示。左边是个性化推荐板块，右边是热门电影板块。电影内容通过 QTableWidget 进行布置。对每张表格通过 setColumnCount 和 setHorizontalHeaderLabels 设置列数和列名之后，再将每部电影的信息通过 setItem 方法写入表格。

豆瓣电影推荐系统

欢迎您！ cabaret

根据您的喜好为您推荐以下电影：

电影名称	主演	电影类型	电影评分	电影链接
1 《湖畔的陌生人》	阿兰·吉罗迪	剧情 情色 同性	7.4	https://...
2 《谋杀》	魏娜·莎德	剧情 悬疑 犯罪	8.3	https://...
3 《林中小屋》	德鲁·高达	恐怖	7.5	https://...
4 《危机边缘》	J·J·艾布拉姆斯	剧情 科幻 悬疑	8.6	https://...
5 《杀人狂魔》	亚历山大·阿嘉	悬疑 恐怖 犯罪	5.8	https://...
6 《都市女孩》	莉娜·邓纳姆	喜剧	7.9	https://...
7 《死亡录像3：...	Luis Berdejo	喜剧 惊悚 恐怖	6.1	https://...
8 《湖畔的陌生人》	阿兰·吉罗迪	剧情 情色 同性	7.4	https://...
9 《谋杀》	魏娜·莎德	剧情 悬疑 犯罪	8.3	https://...
10 《林中小屋》	德鲁·高达	恐怖	7.5	https://...
11 《危机边缘》	J·J·艾布拉姆斯	剧情 科幻 悬疑	8.6	https://...
12 《杀人狂魔》	亚历山大·阿嘉	悬疑 恐怖 犯罪	5.8	https://...
13 《都市女孩》	莉娜·邓纳姆	喜剧	7.9	https://...
14 《死亡录像3：...	Luis Berdejo	喜剧 惊悚 恐怖	6.1	https://...
15 《湖畔的陌生人》	阿兰·吉罗迪	剧情 情色 同性	7.4	https://...
16 《谋杀》	魏娜·莎德	剧情 悬疑 犯罪	8.3	https://...
17 《林中小屋》	德鲁·高达	恐怖	7.5	https://...
18 《危机边缘》	J·J·艾布拉姆斯	剧情 科幻 悬疑	8.6	https://...

热门电影：

电影名称	主演	评分	电影类型	上映时间	电影链接
1 《千与千寻》	宫崎骏	9.4	剧情 动画 奇幻	2019	https://...
2 《昨日的美...	西岛秀俊 内...	8.9	剧情 同性 家庭	2020	https://...
3 《代号基亚...	福山润 野上...	8.7	动画	2019	https://...
4 《灵能百分...	伊藤节生 榎...	9.3	动画	2019	https://...
5 《鬼灭之刃 ...	大冢芳忠 梶...	9.1	动作 动画 惊悚	2019	https://...
6 《吹响悠风...	黑泽朋世 朝...	9.2	动画	2019	https://...
7 《刘若英”陆...	刘若英 陆...	9.3	音乐 真人秀	2020	https://...
8 《青春猪猪...	石川界人 濑...	8.7	剧情 爱情 动画	2019	https://...
9 《bilibili晚...	朱广权 陈超...	9.1	歌舞 真人秀	2019	https://...
10 《多哥》	威廉·达福 朱...	8.8	剧情 冒险	2019	https://...
11 《爱尔兰人》	罗伯特·德尼...	8.9	剧情 传记 犯罪	2019	https://...
12 《波西米亚...	拉米·马雷克 ...	8.7	剧情 同性 音乐	2019	https://...
13 《海蒂和爷...	阿努克·斯特...	9.2	剧情 家庭 冒险	2019	https://...
14 《美丽人生》	罗伯托·贝尼...	9.5	剧情 喜剧 爱情	2020	https://...
15 《小丑》	华金·菲尼克...	8.7	剧情 惊悚 犯罪	2019	https://...
16 《何以以为家	赞恩·阿尔·拉...	9.1	剧情	2019	https://...
17 《寄生虫》	宋康昊 李善...	8.7	剧情 喜剧	2019	https://...
18 《绿皮书》	维果·莫滕森 ...	8.9	剧情 喜剧 传记	2019	https://...
19 《海上钢琴...	蒂姆·罗斯 普...	9.3	剧情 音乐	2019	https://...

图 6.8 用户主界面

Fig. 6.8 Main user interface

点击该按钮进入电影详细信息搜索界面,如图 6.11 所示,通过输入完整的电影名称,用户可以了解电影的详细信息:



图 6.11 电影详细信息搜索界面

Fig. 6.11 Movie details search interface

按下“搜索”按钮后,如图 6.12 所示,显示如下界面:导演、编剧、主演、电影简介等信息都可以看到。



图 6.12 电影详情界面

Fig. 6.12 Movie detail interface

6.2.5 用户个人界面

进入主界面后，有一个“个人主页”按钮，如图 6.13 所示，点击该按钮，简单的个人信息显示如图 6.14 所示：



图 6.13 个人主页按钮

Fig. 6.13 Personal home button

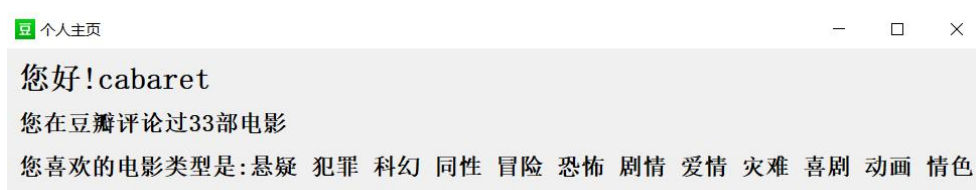


图 6.14 用户个人界面

Fig. 6.14 User personal interface

6.3 系统测试

系统测试是软件开发的一个重要环节，它可以帮助我们评估系统是否满足我们的需求，并发现系统可以进一步完善及扩展的地方。本系统通过黑盒测试法对其进行功能测试。

登录和注册功能测试如表 6.1 所示：

表 6.1 登录和注册功能测试

Table 6.1 Log in and register for functional testing

用例编号	测试步骤	预期结果	测试结果
1	登录页面输入正确信息	登录成功	登录成功跳转主页
2	登录页面输入错误信息	登录失败	提示用户名或密码错误
3	注册页面输入正确信息	注册成功	注册成功跳转主页
4	注册页面输入错误信息	注册失败	提示用户名已被注册

电影模块功能测试如表 6.2 所示:

表 6.2 电影模块功能测试
Table 6.2 Movie module feature test

用例编号	测试步骤	预期结果	测试结果
1	登陆后显示热门电影	显示热门电影列表	成功显示热门电影列表
2	点击电影详细信息	跳转电影详细信息页面	成功跳转电影详细信息页面
3	电影详细信息页面输入完整电影名称搜索	显示搜索的电影信息	成功显示搜索的电影信息
4	电影详细信息页面输入不完整电影名称搜索	查询不到电影信息	错误提示, 找不到相关电影
5	点击电影搜索	跳转电影搜索页面	成功跳转电影搜索页面
6	电影搜索页面输入部分电影名称	搜索电影名中含有输入部分的相关电影信息	成功搜索出电影名中含有输入部分的相关电影信息

个人信息模块功能测试如表 6.3 所示:

表 6.3 个人信息模块功能测试
Table 6.3 Personal information module function test

用例编号	测试步骤	预期结果	测试结果
1	登陆后不提交用户爱好	喜好电影推荐列表显示为空	喜好电影推荐列表显示不显示
2	登陆后正确提交用户爱好	显示电影推荐列表	成功显示电影推荐列表信息
3	点击个人主页	跳转个人主页页面	成功跳转个人主页
4	个人主页信息显示	显示用户名、电影喜好等	成功显示用户名、电影喜好等信息

本章小结

在本章中，实现了一个电影推荐系统。首先，利用数据抓取技术从各类电影相关信息网站中收集电影数据和用户数据，为推荐引擎提供丰富的原始数据基础。其次将收集到的用户数据和电影信息数据进行数据存储。接着对数据可视化功能进行实现，本项目通过 PyQt5 实现 GUI，用户可以登陆该系统进行操作，最终为用户生成推荐电影。

结 论

本文主要研究了基于协同过滤的推荐系统，从理论框架、核心技术到实际应用，再到算法改进与优化，进行了全方位的探讨。论文首先阐述了推荐系统在解决信息过载问题、提升用户体验以及推动电商、影视、新闻、社交等行业个性化服务方面的关键作用。其中，基于用户的协同过滤推荐技术凭借分析用户历史行为和兴趣偏好来实现个性化推荐，但在实际应用中面临数据稀疏性、冷启动问题和计算效率限制等挑战。

针对以上问题，论文详细分析了现有解决方案，并提出了具体的改进措施。在第三章，论文专门讨论了协同过滤中的冷启动与稀疏性问题，并通过对基于用户的协同过滤算法进行实例分析和实验验证，揭示了传统算法的局限性。

随后，在第四章，论文介绍了基于矩阵分解的协同过滤新算法，该算法在计算效率、处理大数据集及易用性方面都有显著提升，实验表明它能有效提高推荐效果。

在第五章，本文提出了融合多层感知机的协同过滤推荐算法-MPCF，这是一种结合深度学习与协同过滤的先进推荐技术，能够有效克服传统算法存在的冷启动问题，提高预测准确性，并展现出良好的可解释性和灵活性。实验部分，在 MovieLens 数据集上验证了改进后的 MPCF 算法的有效性。

第六章，在前面工作基础上，本文以电影推荐为例，设计了基于协同过滤的电影推荐系统，针对目前电影推荐系统很难满足用户的个性化需求，通过该系统，用户只需要提供自己的观影行为和偏好，系统就可以帮助用户快速找到符合自己口味的电影作品，提高用户的观影体验和满意度。

论文整体结构严谨，从推荐系统的起源、发展、分类、算法原理，直至具体技术改进和案例分析，构建了一个完整的理论和实践框架。通过详尽的实验验证和对比分析，展现了对推荐系统技术深入理解和实践应用的成果。

参考文献

- [1] 袁强.一种基于深度强化学习的推荐算法研究[D].云南财经大学,2023.1.
- [2] 项亮. 推荐系统实践 [M].北京:人民邮电出版社, 2012.
- [3] Agarwal G , Bahuguna H , Agarwal A .Solving Cold-Start Problem in Recommender System Using User Demographic Attributes[J]. [2024-06-20].
- [4] Zhang M, Ai X, Hu Y. Chinese Text Classification System on Regulatory Information Based on SVM[C] // IOP Publishing Ltd. 2019 : 022133.
- [5] 程茜萌.基于机器学习的定增项目精准营销研究[D].上海财经大学,2022.9-10.
- [6] Conneau A, Schwenk H, Barrault L, et al. Very Deep Convolutional Networks for Text Classification[J], 2017.
- [7] 刘园园.基于深度学习的农业图书推荐算法研究[D].武汉轻工大学,2022.2-6.
- [8] Ozge Aytekin Karakaya T. Effective methods for increasing aggregate diversity in recommender systems[J]. Knowledge and information systems, 2018, 56(2).
- [9] Kaur H, Kumar N, Batra S. An efficient multi-party scheme for privacy preserving collaborative filtering for healthcare recommender system[J]. Future Generation Computer Systems,2018,86:297 – 307.
- [10] 黄贤英,熊李媛,李沁东.基于改进协同过滤算法的个性化新闻推荐技术[J].四川大学学报(自然科学版),2018,55(01):49-55.
- [11] 张鑫.基于竞争者信息的个性化推荐研究[D].北京理工大学,2017,6-49.
- [12] 丁岚. 基于融合时间因子和项目聚类的概率矩阵分解推荐算法研究 [D].[S.l.]:吉林大学,2-3.
- [13] 刘洋. 基于 SVD 矩阵分解技术和 RkNN 算法的协同过滤推荐算法 [J]. 湖南工程学院学报 (自然科学版),2015,025(001):44 – 47.
- [14] 秦绪涛. 关于铁路货运图书市场拓展的思考 [J]. 新闻研究导刊,2022,13(2):203 – 205.
- [15] 沈西挺, 董智佳.反映用户兴趣变化的协同过滤算法 [J].计算机应用与软件, 2013, 30(6):3.
- [16] Gai L I, Lei L I, Polytechnic S, et al. Collaborative filtering algorithm based on matrix decomposition[J]. Computer Engineering and Applications, 2011.
- [17] 刘吴颖.基于课程画像的在线教育课程推荐方法研究[D].哈尔滨工业大学,2021.
- [18] 杨敬东.铁路企业职工培训问题研究 [J].中文科技期刊数据库 (全文版) 经济管理,00009 – 00009.
- [19] Liang G, Wen J, Zhou W. Individual diversity preference aware neural collaborative filtering[J]. Knowledge-Based Systems, 2022, 258:109730.
- [20] Yan H C, Wang Z R, Niu J Y, et al. Application of covering rough granular computing model in collaborative filtering recommendation algorithm optimization[J]. Advanced Engineering Informatics, 2022, 51 : 101485.
- [21] Ge Z Q, Liu X Y, Li Q, et al. PrivItem2Vec: A privacy-preserving algorithm for top-N recommendation[J]. International Journal of Distributed Sensor Networks, 2021, 17(12) : 13.
- [22] Bao T, Xu L, Zhu L, et al.Privacy-Preserving Collaborative Filtering Algorithm Based on Local Differential Privacy[J].China Communications,2021,18(11):42-60.
- [23] 罗松吟.基于互注意力机制的评论感知推荐算法研究[D].北京交通大学,2021.
- [24] 余池.网络安全等级保护测评方案推荐系统研究[D].湖北大学,2021.
- [25] 周开拓,罗梅,苏璐.智能推荐在新媒体内容分发中的应用[J].人工智能,2020,(02):105-115.

- [26] 朱金金.基于图神经网络和注意力机制的会话推荐方法研究[D].辽宁工程技术大学,2022.
- [27] 陈嘉宁.异构网络中分层缓存与缓存内容推荐算法研究[D].北京邮电大学,2020.
- [28] 许海玲, 吴潇, 李晓东, et al. 互联网推荐系统比较研究 [J]. 软件学报, 2009(2) : 13.
- [29] 张锋, 常会友. 使用 BP 神经网络缓解协同过滤推荐算法的稀疏性问题 [J]. 计算机研究 与发展, 2006, 43(4) : 667 – 672.
- [30] Xue G R, Lin C, Yang Q, et al. Scalable collaborative filtering using cluster-based smooth- ing[C] // SIGIR 2005: Proceedings of the 28th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, Salvador, Brazil, August 15-19, 2005. 2005 : p.114 – 121.
- [31] 汤吴建.基于混合相似度的协同过滤推荐[J].软件导刊,2017,16(02):32-35.
- [32] 郁雪. 基于协同过滤技术的推荐方法研究 [D]. [S.l.] : 天津大学, 2009.
- [33] Mcjones P. EachMovie collaborative filtering data set[J]. dec systems research center, 1997.
- [34] Hang Y, Guiran C, Xingwei W, et al. Application of Multi-Attribute Rating Matrix in Cold- start Recommendation[C] // 2010 Fourth International Conference on Genetic and Evolution- ary Computing. [S.l.] : IEEE, 2010 : 55 – 58.
- [35] KimS, Choi S M, Han Y S, et al. Analyzing item features for cold-start problems in rec- ommendation systems[C] // 2014 Tenth International Conference on Intelligent Information Hiding and Multimedia Signal Processing. [S.l.] : IEEE, 2014 : 167 – 170.
- [36] Liu P, Cao J, Liang X, et al. A two-stage cross-domain recommendation for cold start prob- lem in cyber-physical systems[C] // 2015 International Conference on Machine Learning and Cybernetics (ICMLC) : Vol 2. [S.l.] : IEEE, 2015 : 876 – 882.
- [37] Uyangoda L, Ahangama S, Ranasinghe T. User Profile Feature-Based Approach to Address the Cold Start Problem in Collaborative Filtering for Personalized Movie Recommendation[C] // 2018 Thirteenth International Conference on Digital Information Management (ICDIM). [S.l.] : IEEE, 2018 : 24 – 28.
- [38] Miller BN, AlbertI, Lam S K, et al. MovieLens unplugged: experiences with an occasionally connected recommender system[C] // International Conference on Intelligent User Interfaces. 2003 : p.263 – 266.
- [39] Ryza B S, Laserson U, Owen S, et al. Advanced Analytics with Spark[J], 2015.
- [40] 朱桂明, 宾辰忠, 古天龙, 等. 基于知识图谱的用户偏好神经建模框架[J].模式识别与人工智能 ,2019,32(07):661-668.
- [41] 周玉甲.基于神经网络的入侵检测系统研究[D].湖南大学,2006.
- [42] 龚四相.基于Spark的混合推荐系统分析与实现[D].长江大学,2020.温源.互联网文本信息挖掘与个性化推荐的研究[D].北京交通大学,2014.